人工智能与深度学习

神经网络与深度学习（二）

华中科技大学 机械科学与工程学院
陈伟

2026

改进神经网络

神经网络基础

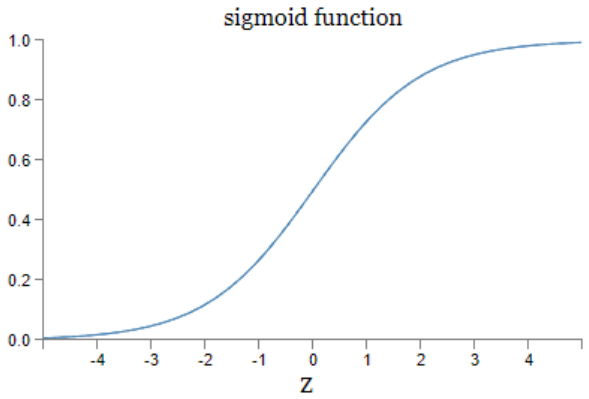
- **我们已经对神经网络进行了基本的介绍**

- **在之前的例子中，我们使用了**
  - 二次损失函数

$$C = \frac{1}{2n} \sum_x ||\boldsymbol{y}(x) - \boldsymbol{a}^L(x)||^2$$

  - Sigmoid 激活函数

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



改进神经网络

- **实际使用中，我们还可以从很多方面来提升神经网络的学习能力**
- **我们重点介绍以下几点**
  - 交叉熵（cross entropy）损失函数
  - Softmax 激活函数与 ReLU 激活函数
  - 过拟合与正则化
    - Dropout
    - 数据增强
  - 权重初始化
  - 超参数（hyper-parameter）

交叉熵（cross-entropy）
损失函数

损失函数

- 回顾：我们之前使用的损失函数如下

$$C = \frac{1}{2n} \sum_x ||\boldsymbol{y}(x) - \boldsymbol{a}^L(x)||^2$$

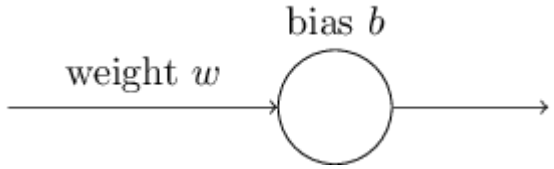
n 是样本总数，求和是针对所有的样本，$\boldsymbol{y}(x)$ 是对样本 x 的标签，$\boldsymbol{a}^L$ 是神经网络最后一层的输出

有没有比这个基于2-范数的损失函数更好的损失函数呢？

示例：神经网络学习速度

- 我们希望神经网络能够学得快。

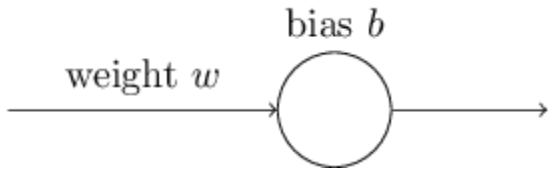
  考虑以下这个只含有一个神经元的例子（激活函数仍然是 Sigmoid）



- 我们希望这个“神经网络”能学到如下的效果：输入为 1 时，输出为 0，即 $x = 1$ 时 $y = 0$

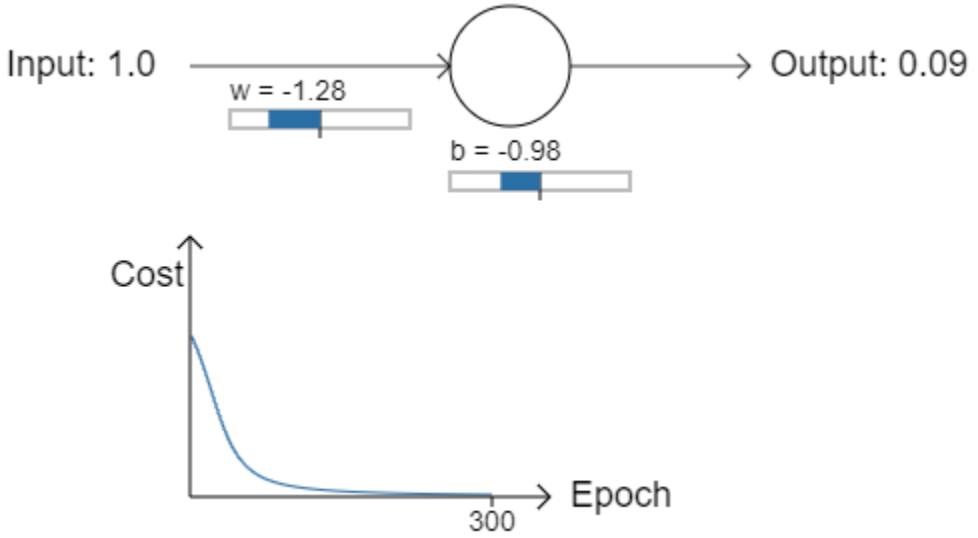
示例：神经网络学习速度

- 数值示例1：初始值取 $w = 0.6, b = 0.9$，那么当输入为 1 时，(经过Sigmoid函数后的)初始输出是 0.82



- 我们使用前文所讲的二次代价函数，把学习速率 η 定为 0.15，使用梯度下降，观察该神经网络需要学习多久才能接近我们希望的结果

示例：神经网络学习速度

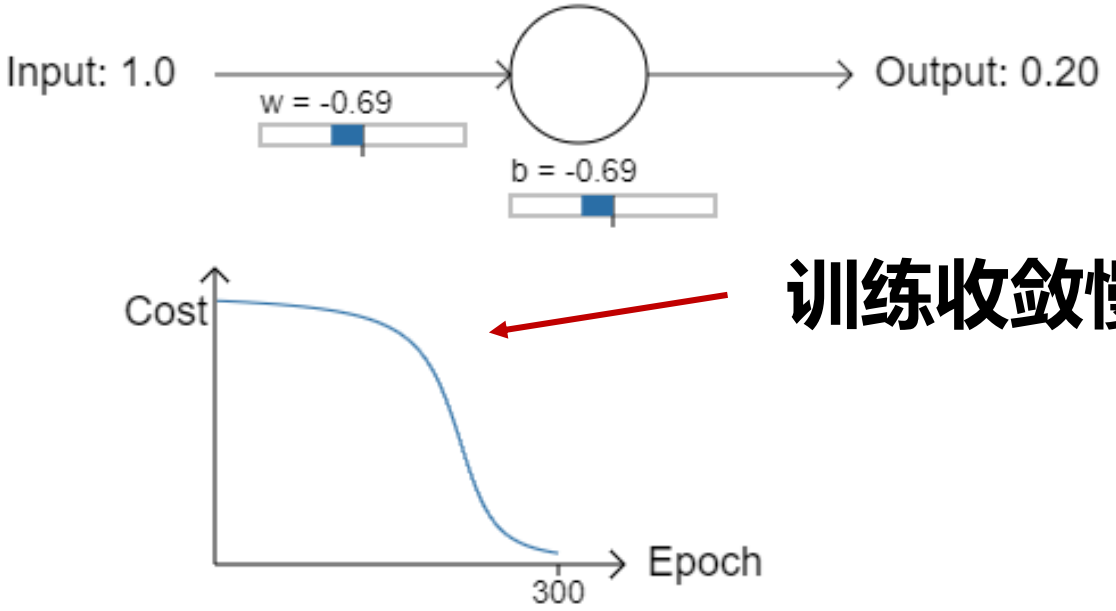


数值示例1的训练收敛情况

- 可以看到，这个神经网络较快地收敛到 $w = -1.28, b = -0.98$，此时输出为 0.09 （注：使“输入=1，输出=0或接近0”的 w 和 b 不唯一）

示例：神经网络学习速度

- 数值示例2：假如我们更改一下初始值，取 $w = 2, b = 2$，对应的初始输出为 0.98；再用同样的方式对神经网络进行训练 $(\mu = 0.15)$



数值示例2的训练收敛情况

神经网络学习速度

- 同一个神经网络，为什么在权重和偏置的初值不同的时候，训练收敛的速率不同？

- 回顾：基于梯度下降的参数更新公式为

$$\boldsymbol{w} \to \quad \boldsymbol{w}' = \boldsymbol{w} - \eta \frac{\partial C}{\partial \boldsymbol{w}}$$

$$b \to \quad b' = b - \eta \frac{\partial C}{\partial b}$$

神经网络学习速度

- 训练的速率和代价函数的偏导，即 $\frac{\partial C}{\partial w}$ 与 $\frac{\partial C}{\partial b}$ 有关

  前述第二个例子中的“学习速率慢”，其原因是 $\frac{\partial C}{\partial w}$ 与 $\frac{\partial C}{\partial b}$ 太小

- 我们来计算一下。训练中使用的二次损失函数为

$$C = \frac{(y-a)^2}{2},$$

  其中 a 是当输入为 $x = 1$ 时神经元的输出，$a = \sigma(z)$ 而 $z = wx + b$，此时期望的输出是 $y = 0$

神经网络学习速度

- 训练中使用的二次损失函数为

$$C = \frac{(y-a)^2}{2}$$

神经元输出为 $a = \sigma(z) = \sigma(wx + b)$

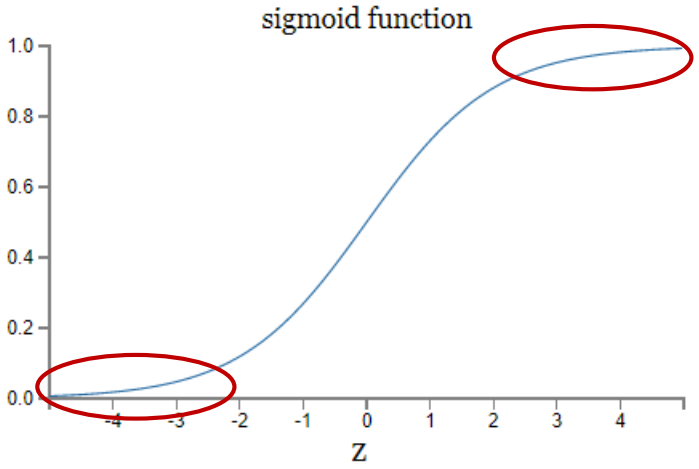
- 根据复合函数求导链式法则，以及$x = 1$，$y = 0$，得到

$$\frac{\partial C}{\partial w} = (a-y)\sigma'(z)x = a\sigma'(z)$$

$$\frac{\partial C}{\partial b} = (a-y)\sigma'(z) = a\sigma'(z)$$

神经网络学习速度

- $$\frac{\partial C}{\partial w} = (a-y)\sigma'(z)x = a\sigma'(z)$$
$$\frac{\partial C}{\partial b} = (a-y)\sigma'(z) = a\sigma'(z)$$

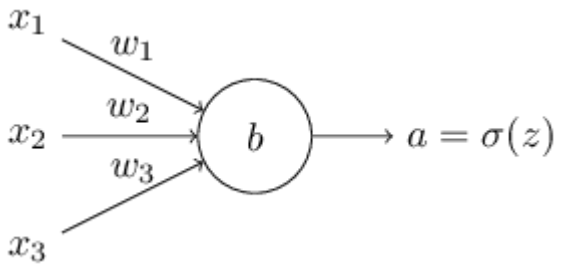


- 当 $\sigma(z)$ 的值接近 0 或 1 时，其曲线非常平坦，其导数 $\sigma'(z)$ 很小

  这会导致在使用梯度下降时，学习速率慢；这一现象也被称为“梯度消失”

交叉熵损失函数（cross-entropy loss function）

- 使用交叉熵损失函数代替二次损失函数，可以一定程度上缓解上述问题
- 考虑如下有多个输入的神经元



$$a = \sigma(z)$$

其中

$$z = \sum_j w_j x_j + b$$

交叉熵损失函数（cross-entropy loss function）

- 定义交叉熵损失函数

$$C = -\frac{1}{n}\sum_{x}[y \ln a + (1-y)\ln(1-a)]$$

其中 n 是训练样本数，求和是针对所有训练样本 x，y 是相应的标签

如何理解交叉熵损失函数

- $$C = -\frac{1}{n}\sum_x [y \ln a + (1-y)\ln(1-a)]$$

- 我们主要考虑 y 的取值为 0 或 1 的情况（如分类问题）；相关讨论可以拓展到 $y \in [0,1]$ 的情况

- $a = \sigma(z) \in (0,1)$，因此，$\ln a < 0$, $\ln(1-a) < 0$

  因此，$C > 0$，即交叉熵总是正的

如何理解交叉熵损失函数

- $$C = -\frac{1}{n}\sum_x [y \ln a + (1-y)\ln(1-a)]$$

- 当神经元的输出 a 接近 y 时，交叉熵的取值接近 0.

  假如 $y = 0,\ a \approx 0$，$\ln(1-a) \approx 0$;

  假如 $y = 1,\ a \approx 1$，$\ln a \approx 0$;

- 当神经元的输出 a 与 y 差别大时，交叉熵的取值也大.

- **交叉熵具有我们期望的损失函数的基本性质**

交叉熵损失函数如何影响收敛速度

- $$C = -\frac{1}{n}\sum_x [y \ln a + (1-y)\ln(1-a)]$$

- 我们计算偏导数

$$\begin{aligned}\frac{\partial C}{\partial w_j} &= -\frac{1}{n}\sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)}\right)\frac{\partial \sigma}{\partial w_j} \\ &= -\frac{1}{n}\sum_x \left(\frac{y}{\sigma(z)} - \frac{1-y}{1-\sigma(z)}\right)\sigma'(z)x_j \\ &= \frac{1}{n}\sum_x \frac{\sigma'(z)x_j}{\sigma(z)(1-\sigma(z))}(\sigma(z)-y)\end{aligned}$$

交叉熵损失函数如何影响收敛速度

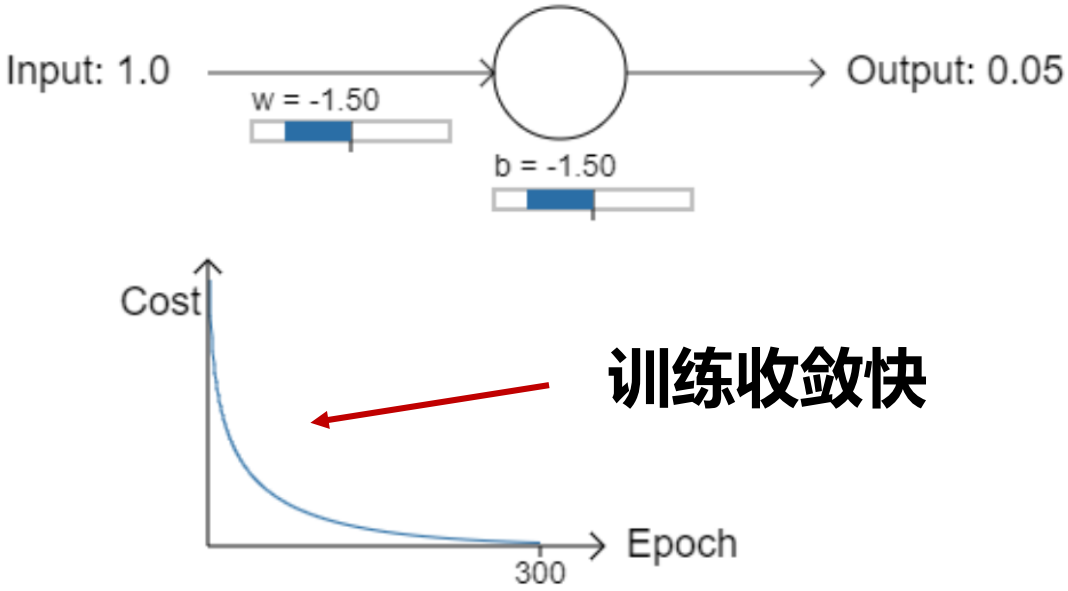
- Sigmoid 函数的定义是 $\sigma(z) = 1/(1+e^{-z})$，因此不难得出 $\sigma'(z) = \sigma(z)(1-\sigma(z))$
- 于是偏导数可以继续化简

$$\begin{aligned}\frac{\partial C}{\partial w_j} &= \frac{1}{n}\sum_x \frac{\sigma'(z)x_j}{\sigma(z)(1-\sigma(z))}(\sigma(z)-y) \\ &= \frac{1}{n}\sum_x x_j(\sigma(z)-y)\end{aligned}$$

$(\sigma(z)-y)$ 是模型的误差，误差越大，该偏导越大，学习速率会相应地越快!

示例：神经网络学习速度

- 在同样的例子中，使用交叉熵损失函数的收敛情况 $w = 2, b = 2$



数值示例2的训练收敛情况：
使用交叉熵损失函数

关于收敛速度的“快与慢”

- **严格地讲，训练收敛速度的快慢与许多因素有关**

> 比如，学习速率 η 就会直接影响训练收敛速率；而 η 的选择与代价函数的绝对大小有关。上页的交叉熵例子中，我们使用了 $\eta = 0.005$

- **本小节中我们说的收敛速度，不是指绝对的收敛速度，而是相对的收敛速度；具体而言，是参数w和b远离其最优值时的收敛速度 vs 它们接近其最优值时的收敛速度**

交叉熵是怎么来的

$$C = -\frac{1}{n}\sum_x [y \ln a + (1-y)\ln(1-a)]$$

- **交叉熵的概念来自信息论领域**

简单来说，假入我们认为 a 是某个随机变量 Y 取值为 1 的概率，$1-a$ 是该随机变量 Y 取值为 0 的概率；那么交叉熵描述了当我们观测到一个实际的 y 值之后的“意外程度”

在 PyTorch 中使用交叉熵损失函数

CROSSENTROPYLOSS

CLASS torch.nn.CrossEntropyLoss(*weight=None, size_average=None, ignore_index=- 100, reduce=None, reduction='mean', label_smoothing=0.0*) [SOURCE]

This criterion computes the cross entropy loss between input logits and target.

It is useful when training a classification problem with C classes. If provided, the optional argument `weight` should be a 1D *Tensor* assigning weight to each of the classes. This is particularly useful when you have an unbalanced training set.

```
import torch.nn as nn
……

criterion = nn.CrossEntropyLoss()
……

loss = criterion(outputs, labels)
```

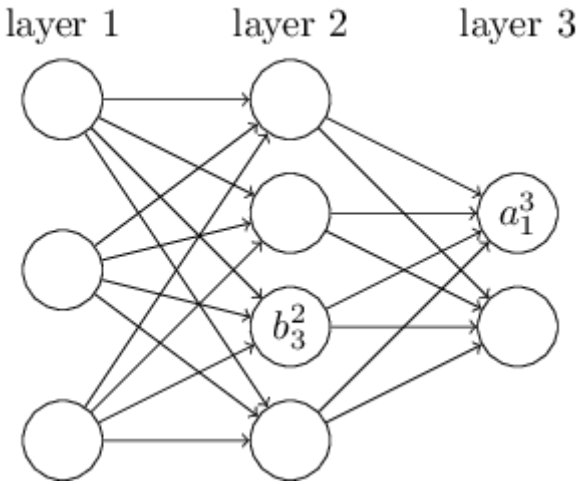
Softmax 激活函数

激活函数

- 激活函数作用在神经网络

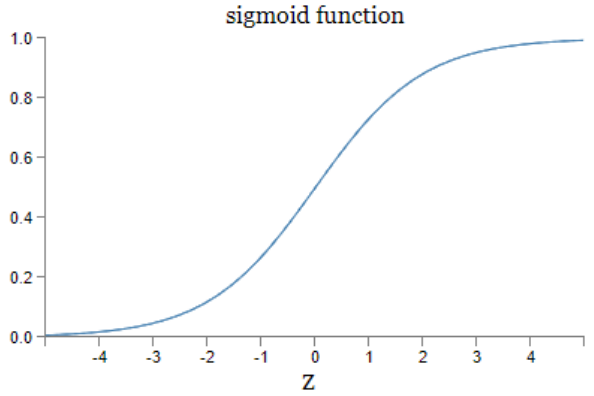
$$z_j^l = \sum_k w_{jk}^l a_k^{l-1} + b_j^l$$

$$a_j^l = \sigma(z_j^l)$$



- 我们之前使用了 Sigmoid 激活函数

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



Softmax 激活函数

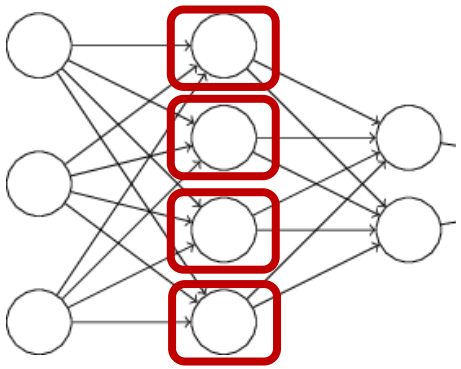
- 与 Sigmoid 函数不同，Softmax 激活函数需要用到一层所有的 $z_j^l, j = 1, 2, ...$ 来计算 a_j^l，即

$$z_j^l = \sum_k w_{jk}^l a_k^{l-1} + b_j^l$$

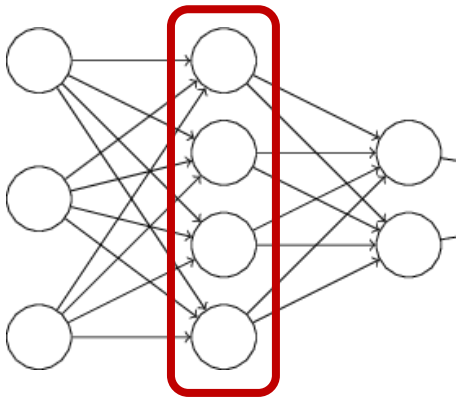
$$\boldsymbol{a}^l = \sigma(\boldsymbol{z}^l)$$

其中，$\boldsymbol{a}^l$ 是第 l 层所有神经元输出组成的向量，$\boldsymbol{z}^l$ 是第 l 层所有 z_j^l 组成的向量

作用于单个神经元的激活函数（如 Sigmoid）



作用于一层神经元的激活函数（如 Softmax）



Softmax 激活函数

- 使用 Softmax 激活函数时，第 l 层神经元第 j 个输出为

$$a_j^l = \frac{e^{z_j^l}}{\sum_k e^{z_k^l}}$$

分母是该层所有神经元在激活函数前的输出，通过 $e^{(\cdot)}$ 之后的和

$$z = [1, 1, 1, 1] \Rightarrow \boldsymbol{a} = [0.25, 0.25, 0.25, 0.25]$$
$$z = [1, 1, 1, 0.5] \Rightarrow \boldsymbol{a} \approx [0.277, 0.277, 0.277, 0.168]$$
$$z = [1, 1, 1, -1] \Rightarrow \boldsymbol{a} \approx [0.319, 0.319, 0.319, 0.043]$$
$$z = [1, 1, 1, 3] \Rightarrow \boldsymbol{a} \approx [0.103, 0.103, 0.103, 0.69]$$

Softmax 激活函数

- $$a_j^l = \frac{e^{z_j^l}}{\sum_k e^{z_k^l}}$$

不难看出，softmax 层输出的向量各元素之和为 1

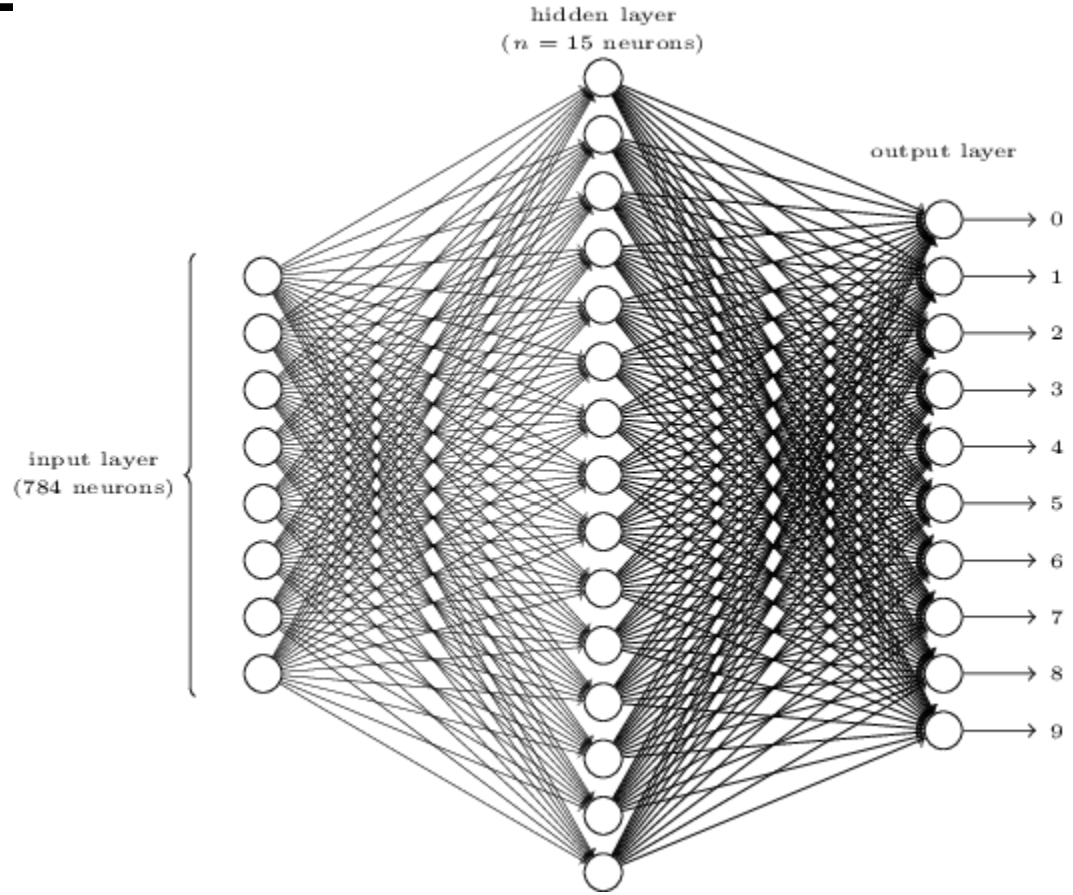
$$\sum_j a_j^l = \frac{\sum_k e^{z_k^l}}{\sum_k e^{z_k^l}} = 1$$

softmax 层把属于 $(-\infty, +\infty)$ 的 z_j^l 转化为 $(0, 1)$ 的 a_j^l，且保留了数值的大小顺序

当我们希望输出的向量被解释为某种“概率”的时候，可以考虑使用 softmax 激活函数

回顾：MNIST手写数字识别神经网络的输出层

- 输出层有10个神经元，其中若第一个神经元被激活了，其输出接近1，则我们认为这个数字是0，以此类推



回顾：MNIST手写数字识别神经网络的输出层

- 若 x 代表的图片是数字 6，那么我们期望的输出是 $y(x) = [0, 0, 0, 0, 0, 0, 1, 0, 0, 0]^T$.

- 我们把 x 经过神经网络后的实际输出记为 a，$a \in \mathbb{R}^{10}$.
  a 可以会出现不完全等于0或1的元素，
  如 $a = [0, 0.1, 0, 0, 0, 0.01, 0, 0.85, 0, 0]$.

- 此时输出层若使用 $\mathrm{softmax}$ 激活函数，输出向量可解释为输入图像识别为某数字的概率

Log-likelihood 损失函数

- Softmax 也可以一定程度上缓解训练速度慢的问题。

  为了解释这一点，我们考虑一个新的损失函数：log-likelihood 损失函数

$$C = -\ln a_y^L$$

  其中 a_y^L 是输出层向量中的一个元素，对应样本的实际标签 y。比如，输入的手写图像是 7，那么 log-likelihood 损失函数就是 $-\ln a_7^L$

Log-likelihood 损失函数

- $$C = -\ln a_y^L$$

  log-likelihood 损失函数一般要求神经网络的输出向量 $\boldsymbol{a}^L$ 可被解释成概率分布

  假如输入图片的实际标签是7，神经网络判断其为 7 的概率也比较大（即 $\ln a_7^L$ 很接近1），相应的log-likelihood代价 C 就很小；反之，则 $\ln a_7^L$ 接近 0， C 则较大。

Log-likelihood 损失函数与 softmax 激活函数

- 为了分析参数训练的速率，我们计算 $\frac{\partial C}{\partial w_{jk}^l}$ 和 $\frac{\partial C}{\partial b_j^l}$

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1}(a_j^l - y_j)$$

$$\frac{\partial C}{\partial b_j^l} = a_j^l - y_j$$

其中 $\boldsymbol{y}$ 是理想的神经网络输出（注：神经网络的输出是一个向量，该向量表示一个概率分布），即在实际标签 j 处 $y_j = 1$，其它元素 $y_i = 0$。

Log-likelihood 损失函数与 softmax 激活函数

- 很多时候，Sigmoid 作为最后一层的激活函数与交叉熵损失函数组合，或 softmax 最后一层的激活函数与 log-likelihood 或交叉熵损失函数组合，都在神经网络中有较好的效果

  如果需要将输出解释为概率分布，一般建议用 softmax 激活函数

在 PyTorch 中使用 softmax

SOFTMAX

CLASS torch.nn.Softmax(*dim=None*) [SOURCE]

Applies the Softmax function to an n-dimensional input Tensor rescaling them so that the elements of the n-dimensional output Tensor lie in the range [0,1] and sum to 1.

Softmax is defined as:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

When the input Tensor is a sparse tensor then the unspecifed values are treated as `-inf`.

```
class NeuralNet(nn.Module):
    def __init__(self):
        ……
        self.Softmax = nn.Softmax()
        ……
    def forward(self, x):
        out = self.fc1(x)
        out = self.Softmax(out)
        return out
```

ReLU (Rectified Linear Unit)
激活函数

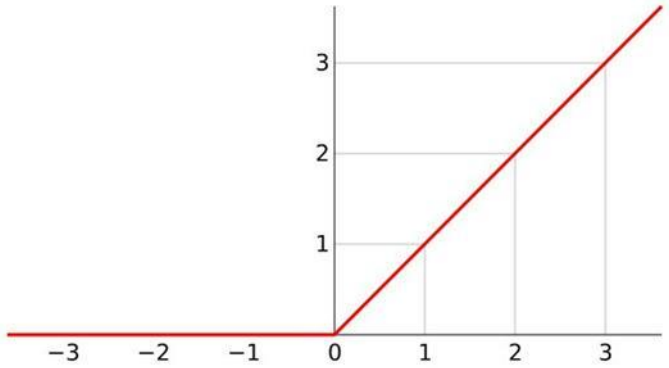
ReLU (Rectified Linear Unit) 激活函数

- ReLU（整流线性单元）也是常用的激活函数

$$f(x) = \max(0, x)$$

也可以写成如下等价形式

$$f(x) = \begin{cases} 0, & \text{if } x \leq 0 \\ x, & \text{if } x > 0 \end{cases}$$

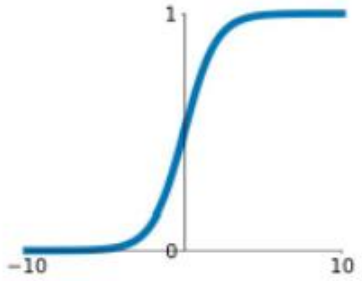


几种激活函数对比

-

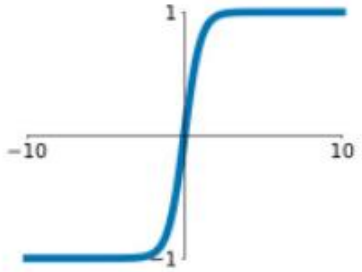
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



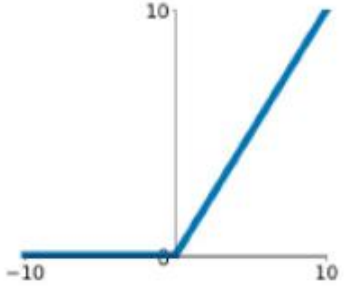
tanh

$$\tanh(x)$$

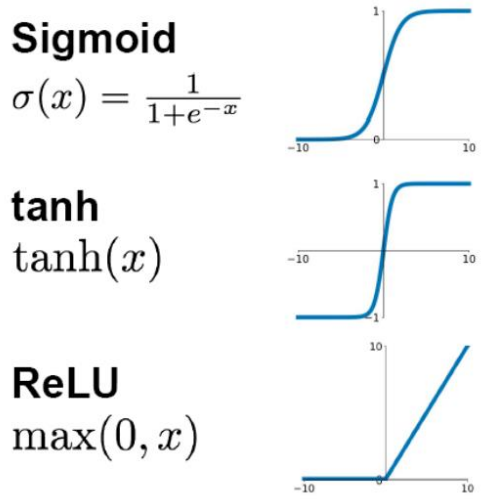


ReLU

$$\max(0, x)$$



ReLU 激活函数

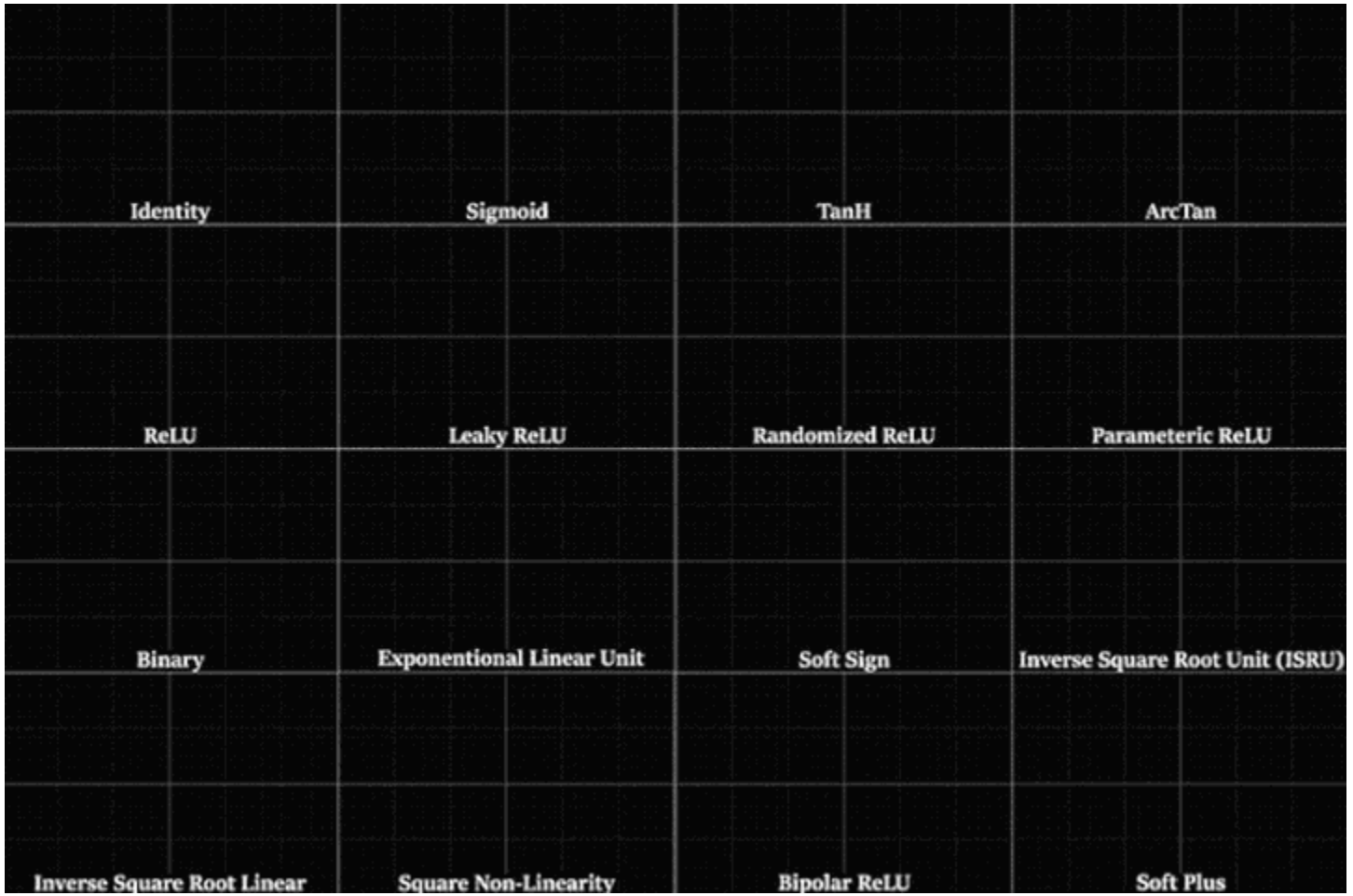


- ReLU 的优点
  - 计算量小
  - 一定程度上缓解“梯度消失”问题
  - 使部分神经元的输出为零，造成网络的“稀疏性”，一定程度上可以缓解过拟合

ReLU 也更符合生物学的神经激活机制

很多时候，ReLU 代替 Sigmoid 可以取得更好的效果

其它激活函数

- 

注：图中第一个（Identity）通常不能直接用作激活函数

PyTorch 中的激活函数

- [torch.nn — PyTorch 1.13 documentation]

TORCH.NN

These are the basic building blocks for graphs:

torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization

`nn.MultiheadAttention`	Allows the model to jointly attend to information from different representation subspaces as described in the paper: Attention Is All You Need.
`nn.PReLU`	Applies the element-wise function:
`nn.ReLU`	Applies the rectified linear unit function element-wise:
`nn.ReLU6`	Applies the element-wise function:
`nn.RReLU`	Applies the randomized leaky rectified liner unit function, element-wise, as described in the paper:
`nn.SELU`	Applied element-wise, as:
`nn.CELU`	Applies the element-wise function:
`nn.GELU`	Applies the Gaussian Error Linear Units function:
`nn.Sigmoid`	Applies the element-wise function:

过拟合与正则化

过拟合

- 和其它机器学习模型一样，神经网络模型也会出现过拟合的问题

- 过拟合：模型在训练集上表现很好，但在测试集上表现明显不如训练集（即泛化能力差）

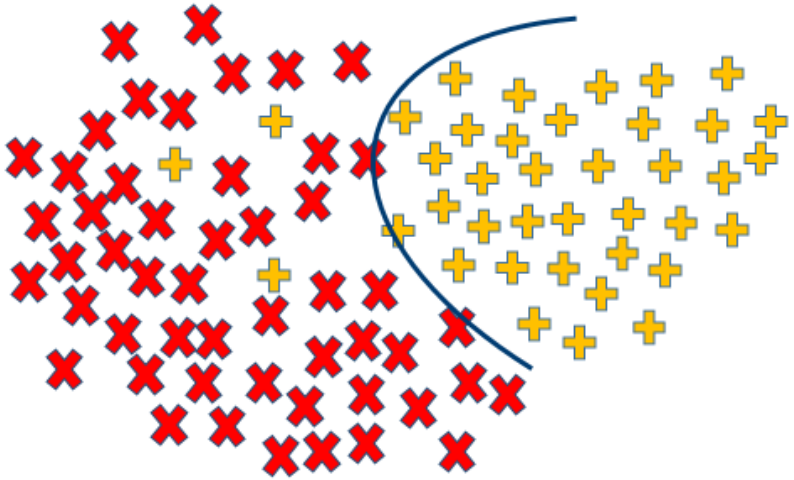
回顾：机器学习模型的过拟合

- 我们希望模型能从训练数据集上学到适用于所有潜在样本的“普遍规律”

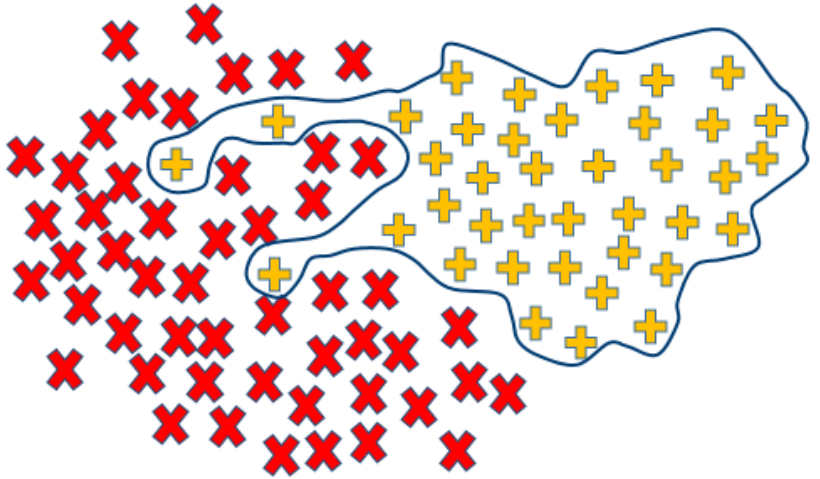
- 然而，在某些情况下，模型可能把样本自身一些特点误当成了一般的性质；这会导致泛化性能下降，这种现象被称为“过拟合”

回顾：机器学习模型的过拟合

- 分类模型：正常拟合和过拟合

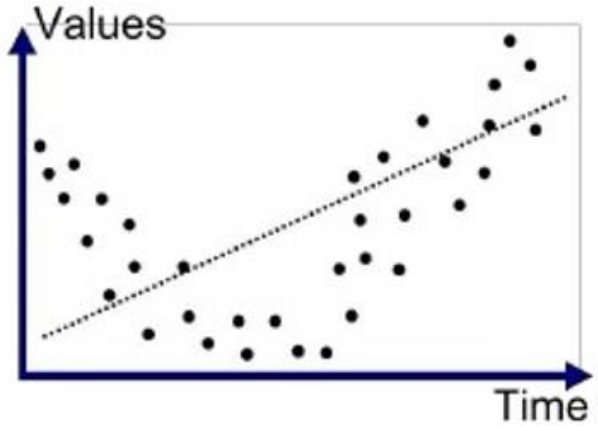


Right !

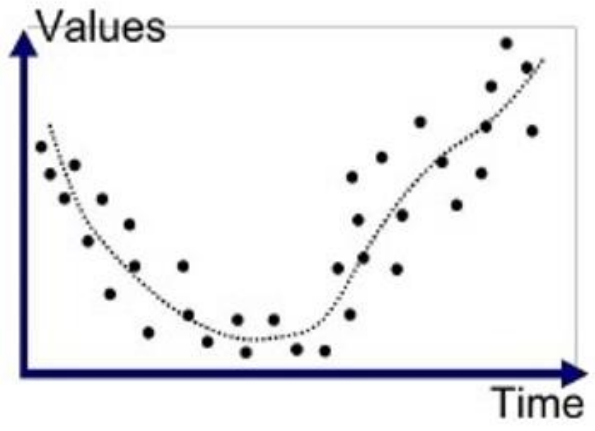


Overfitting

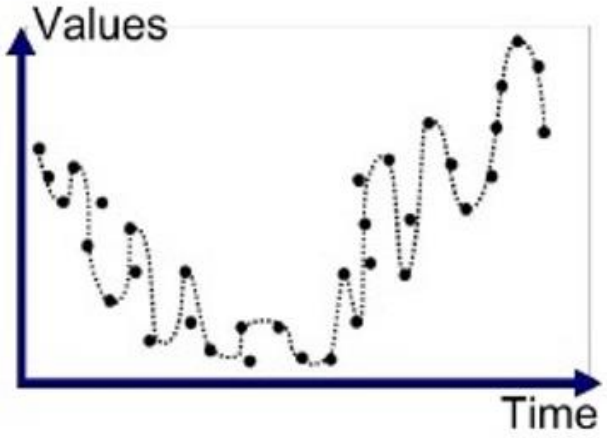
回顾：机器学习模型的过拟合



Underfitted



Good Fit/Robust

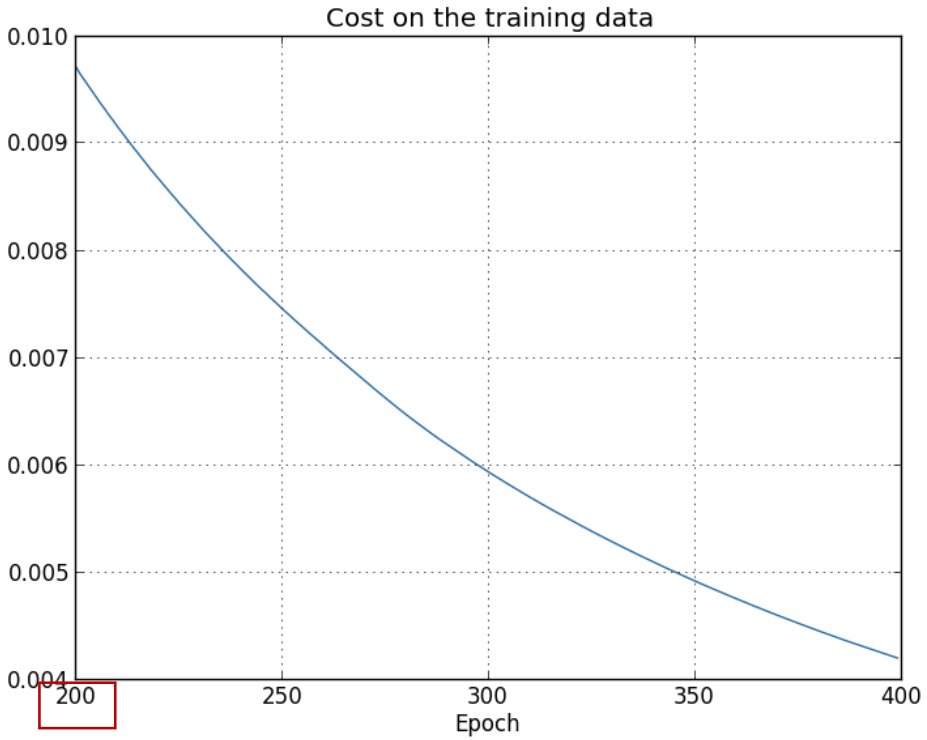


Overfitted

神经网络模型过拟合

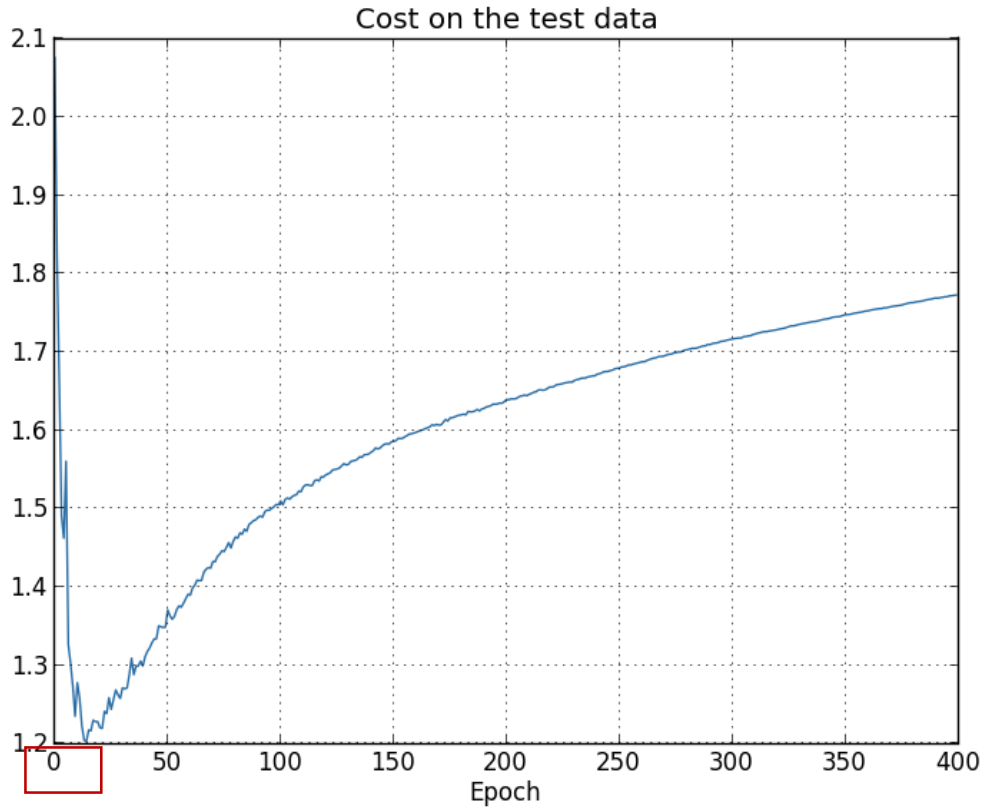
- **使用30个神经元的隐藏层，只用MNIST训练集中的前1000张来训练**

使用交叉熵损失函数，学习速率 $\eta = 0.5$，mini-batch大小为 10



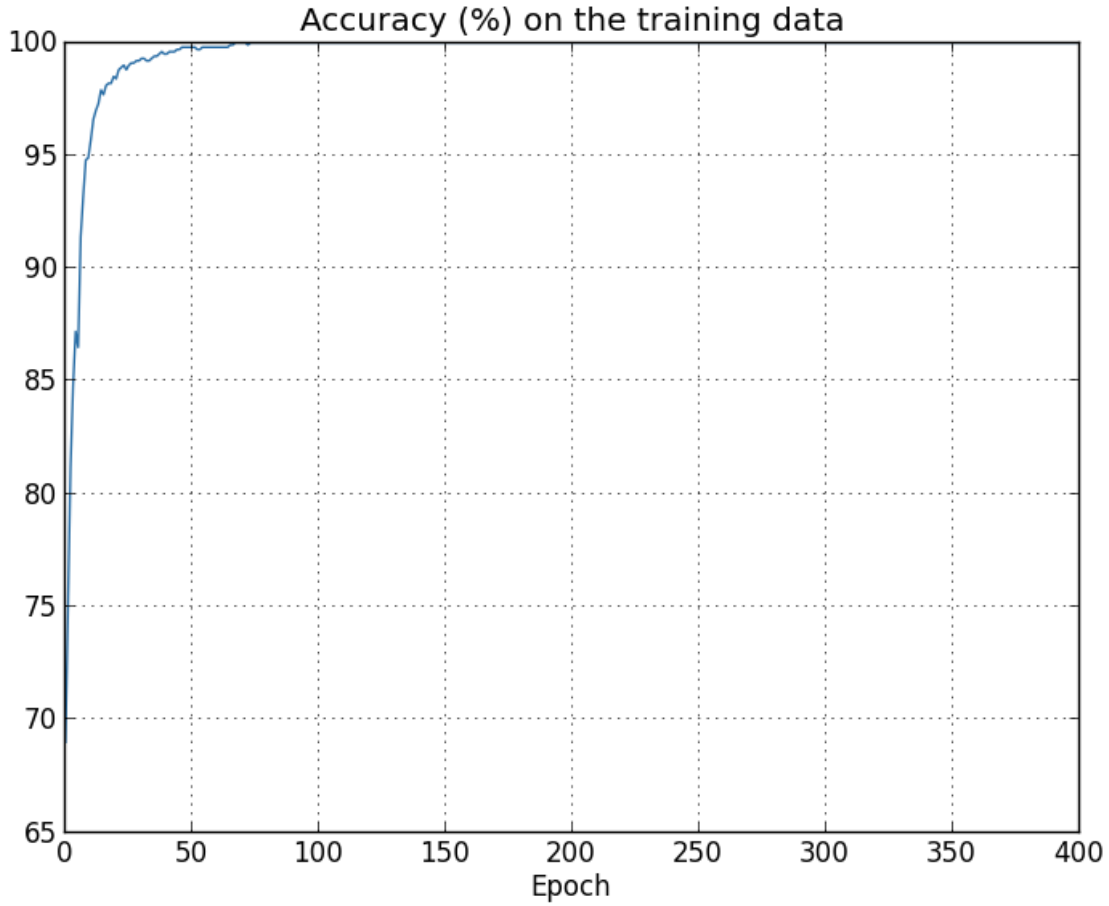
神经网络模型过拟合

- **在测试集上的损失函数（交叉熵）**
  - 最小损失出现在第 15 Epoch



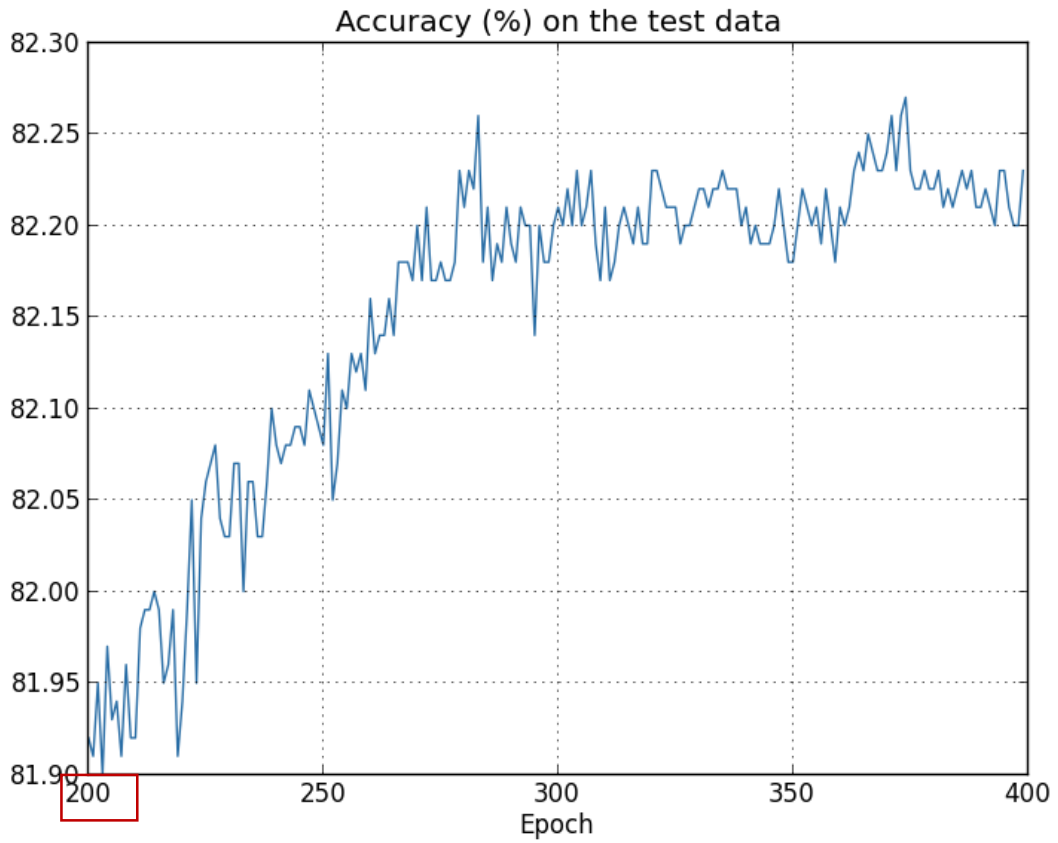
神经网络模型过拟合

- 在训练集上的准确率



神经网络模型过拟合

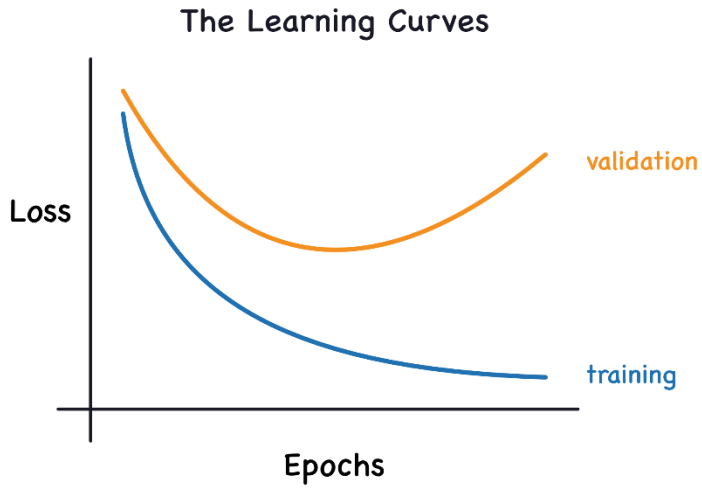
- **在测试集上的准确率**
  - 第 280 Epoch 之后，准确率就基本上不再提升了



神经网络模型过拟合

- 过拟合的表现：模型在验证/测试集上的效果明显差于在训练集上的效果

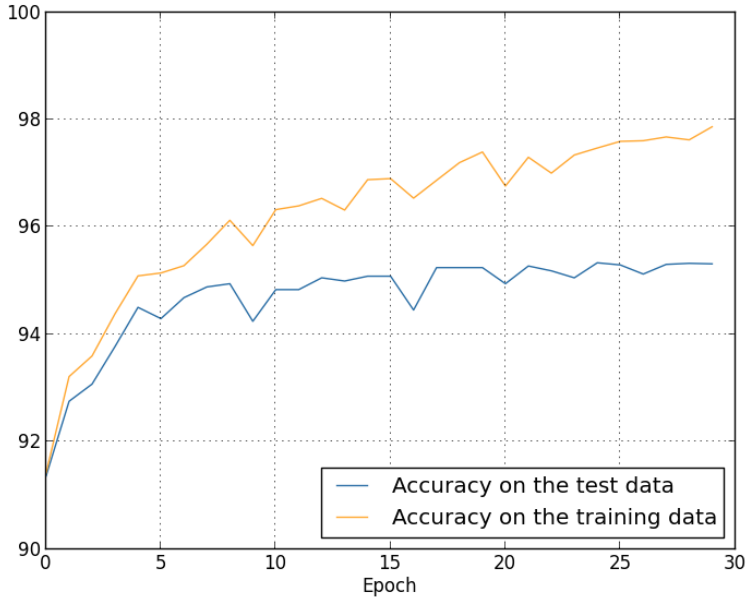
- 过拟合的表现：在训练过程中（Epoch 1 to 400），模型在训练集上效果越来越好，在验证/测试集上没有变化，甚至变得更差



神经网络模型过拟合

- **如何在训练过程中就知道是否过拟合?**
  - 使用验证集（validation dataset），在训练过程中，比较模型在训练集上的效果，和在验证集上的效果

- **使用50000张训练集，过拟合仍然存在，但弱了很多**



神经网络模型参数数量

- 一般情况下，模型参数越多，越容易出现过拟合
- 对手写数字分类的神经网络模型，若含30个隐藏神经元，总参数将有近24000个；若含100个隐藏神经元，总参数将达近80000个
- 一般的深度神经网络可有几百万个参数；最大的深度神经网络模型可有千亿级别的参数
- 如何防止过拟合?

神经网络模型的正则化

- **L2 正则化（L2 regularization）的方法，在神经网络模型中也可使用**
  - 在损失函数中，加入一项参数的2-范数，通常可以有效降低过拟合
  - 如，在交叉熵损失函数加入正则化项

$$C = -\frac{1}{n}\sum_x [y \ln a + (1-y)\ln(1-a)] + \frac{\lambda}{2n}\sum_w w^2$$

  - 在二次损失函数中加入正则化项

$$C = \frac{1}{2n}\sum_x ||y(x) - a^L(x)||^2 + \frac{\lambda}{2n}\sum_w w^2$$

神经网络模型的过拟合

- 一般来说，对未含正则化项的代价函数 C_0，我们都可以通过加入正则化项，来降低训练中的过拟合。

$$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

需要注意的是，正则化项只包含了 w，没有包含偏置项 b

神经网络模型的过拟合

- $$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

直观上，正则化项使神经网络更倾向于学习小的权重w。大的权重只有在显著提升神经网络效果的时候，才“值得”。

回顾：线性回归正则化（Lasso回归与Ridge回归）

- $$(w^*, b^*) = \arg\min_{(w,b)} \left[\sum_{i=1}^{m} (y_i - \boldsymbol{w}^T \boldsymbol{x}_i - b)^2 + \lambda \sum_{j}^{d} |w_j| \right]$$

$\lambda \sum_j^d |w_j|$ 就是 $L1$ 正则化项

- **Lasso 回归对学得的线性模型有什么影响?**
  - 加入正则化项之后，总体来讲，模型对于每个单一特征 x_i 的“敏感程度”降低，因此抑制了过拟合

- 常用的正则化项还有 $L2$ 正则化项 $\lambda \sum_j^d w_j^2$；带 $L2$ 正则化项的线性回归称为 Ridge 回归

L2 正则化

- $$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

我们来看看加入正则化项之后，随机梯度下降算法中使用的梯度有什么变化

$$\frac{\partial C}{\partial w} = \frac{\partial C_0}{\partial w} + \frac{\lambda}{n} w$$

$$\frac{\partial C}{\partial b} = \frac{\partial C_0}{\partial b}$$

L2 正则化

- 梯度下降中，偏置项 b 的学习方式没有变化

$$b \to b - \eta \frac{\partial C_0}{\partial b}$$

对于权重 w 在学习中的更新变成了

$$w \to w - \eta \frac{\partial C_0}{\partial w} - \frac{\eta \lambda}{n} w = \left(1 - \frac{\eta \lambda}{n}\right) w - \eta \frac{\partial C_0}{\partial w}$$

这就等效于在梯度下降更新时，我们对 w 增加了一个“衰减因子” $\left(1 - \frac{\eta \lambda}{n}\right)$

PyTorch 中使用 L2 正则化

- **PyTorch 中使用 L2 正则化，就是在优化器中添加 weight decay（权重衰减）**

SGD

CLASS torch.optim.SGD(*params*, *lr=<required parameter>*, *momentum=0*, *dampening=0*, *weight_decay=0*, *nesterov=False*, ***, *maximize=False*, *foreach=None*, *differentiable=False*) [SOURCE]

Implements stochastic gradient descent (optionally with momentum).

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
μ (momentum), τ (dampening), *nesterov*, *maximize*

for $t = 1$ **to** ... **do**
  $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$
  if $\lambda \neq 0$
    $g_t \leftarrow g_t + \lambda\theta_{t-1}$
  if $\mu \neq 0$
    if $t > 1$
      $\mathbf{b}_t \leftarrow \mu\mathbf{b}_{t-1} + (1-\tau)g_t$

在 PyTorch 中使用 L2 正则化

- **PyTorch 中使用 L2 正则化，就是在优化器中添加 weight decay（权重衰减）**

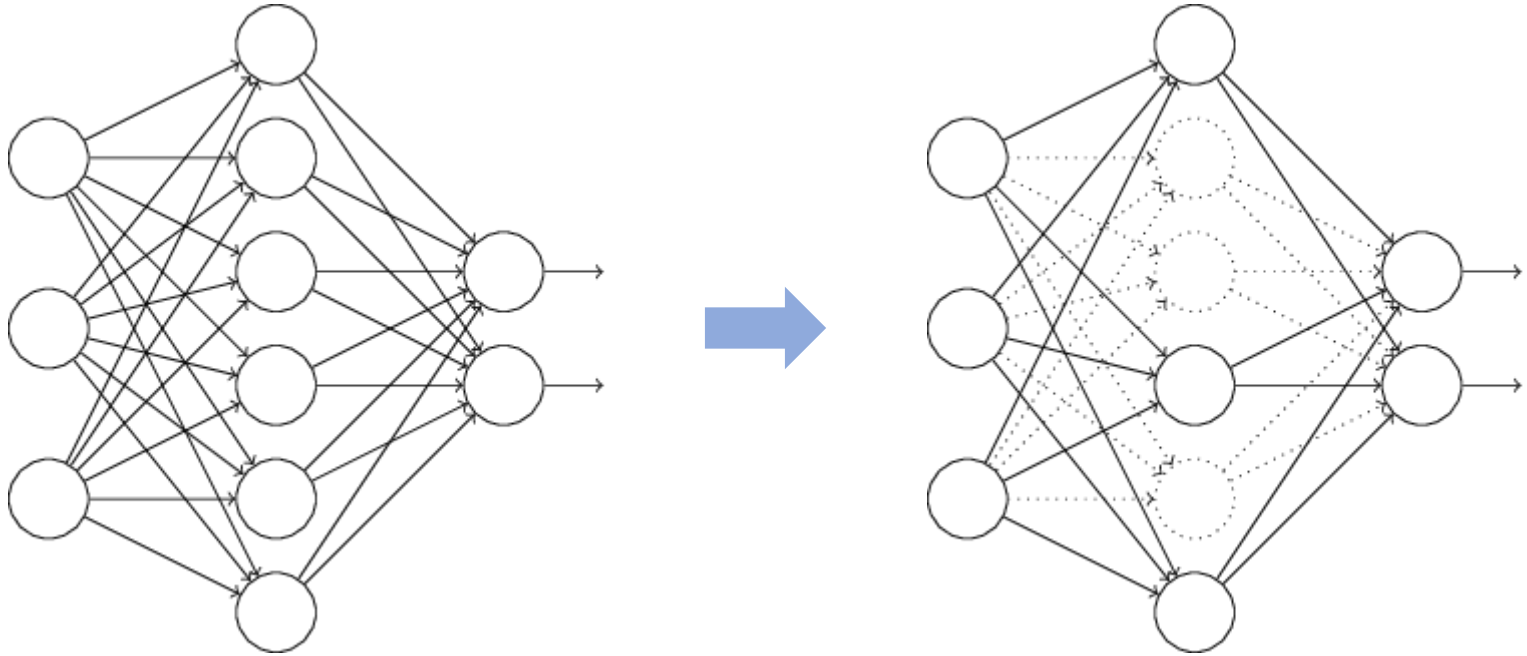
Parameters:

- **params** (*iterable*) – iterable of parameters to optimize or dicts defining parameter groups
- **lr** (*float*) – learning rate
- **momentum** (*float*, *optional*) – momentum factor (default: 0)
- **weight_decay** (*float*, *optional*) – weight decay (L2 penalty) (default: 0)

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, weight_decay=1e-4)
```

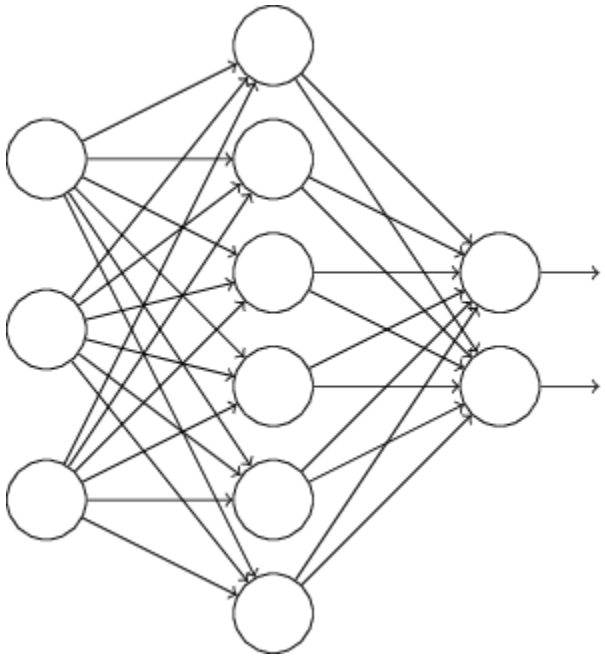
用 Dropout 防止过拟合

- Dropout 是另外一类在神经网络训练中防止过拟合的方法



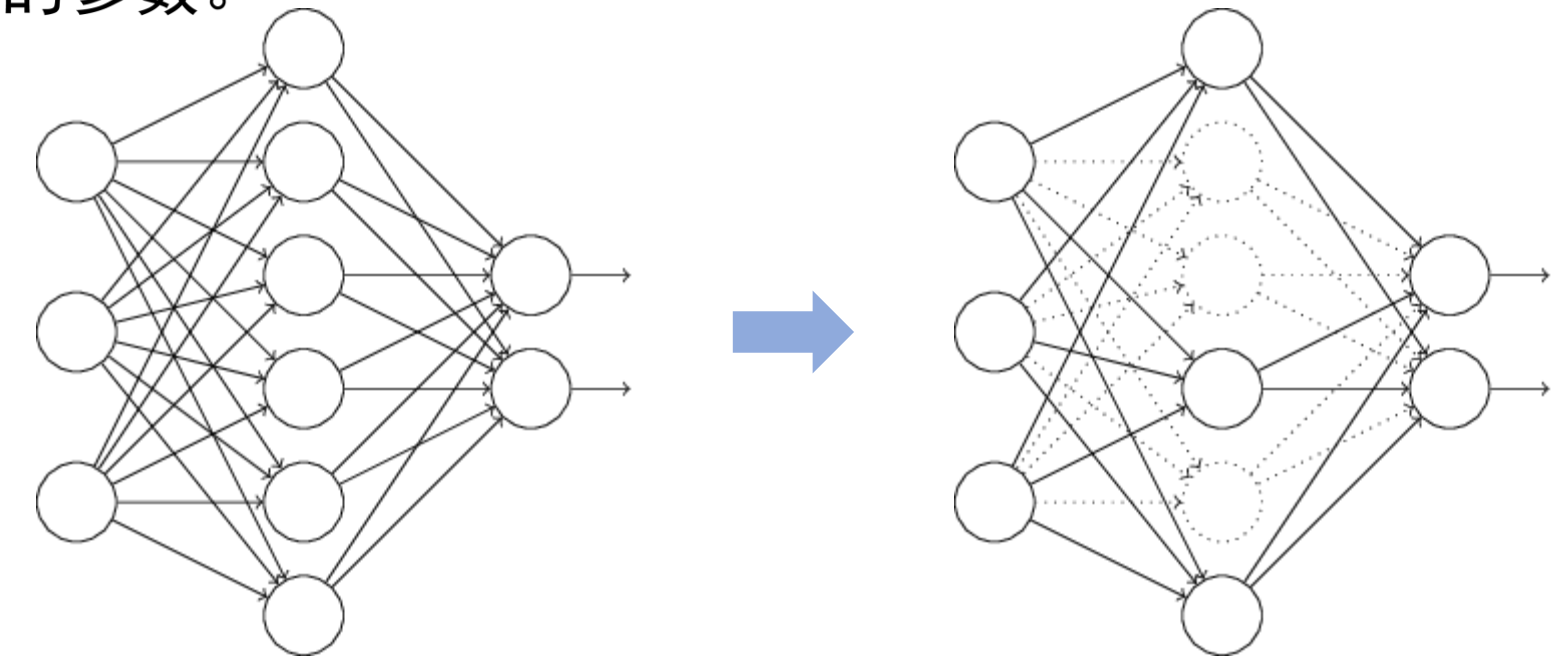
用 Dropout 防止过拟合

- 一般情况下，在神经网络训练中我们让输入 x 经过完整的神经网络计算输出，再通过反向传播算法计算梯度，使用梯度下降更新参数。



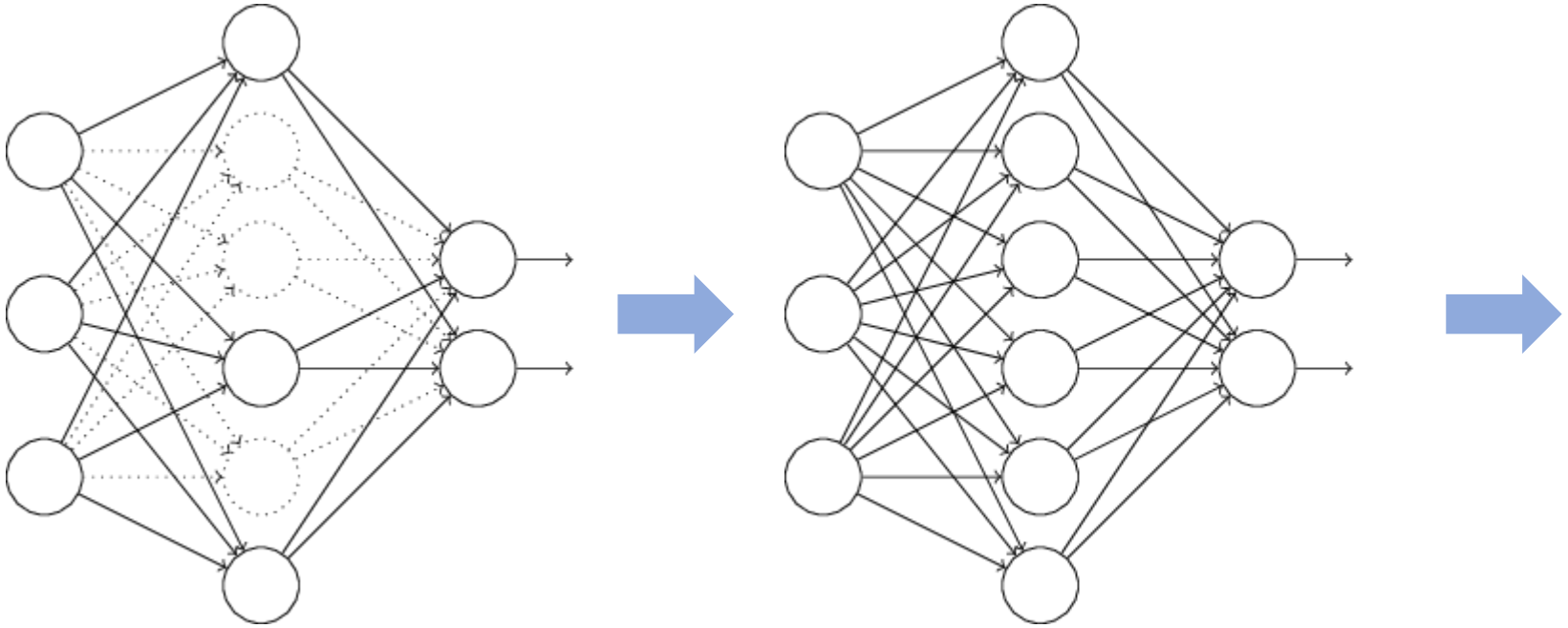
用 Dropout 防止过拟合

- 在使用 dropout 的神经网络训练中，我们会随机地、暂时地删除一半的神经元，得到一个新的神经网络。我们让输入 x 经过这个新的神经网络计算输出，再通过反向传播算法计算梯度，使用梯度下降更新这个新神经网络的参数。



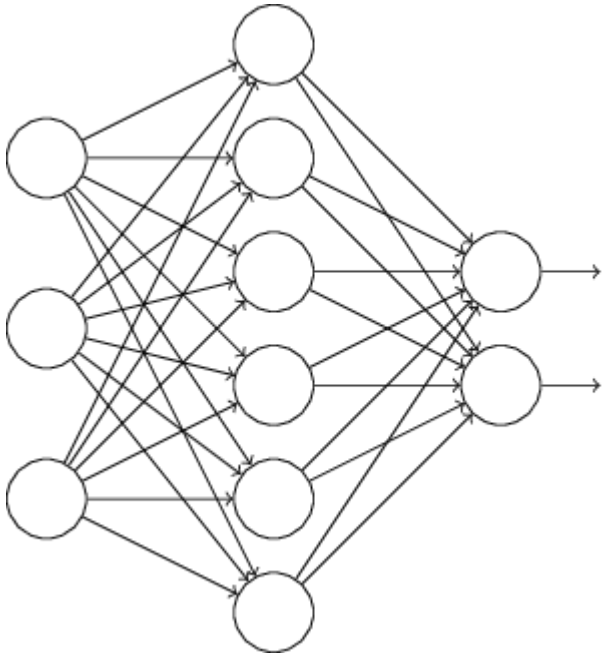
用 Dropout 防止过拟合

- 接下来，我们会把删除的神经元恢复，再次得到一个完整的神经网络。再随机暂时删除一半的神经元，重复刚才的训练过程。训练后，再恢复被删除的神经元。



用 Dropout 防止过拟合

- 由于每次训练都只用了一半的神经元，当把被删除的另一半神经元恢复回来时，完整神经网络中每个神经元的加权输出会是训练时的两倍。因此最后，我们需要将所有的权重都除以2，得到最终神经网络的权重。



用 Dropout 防止过拟合

- Dropout 和集成学习有些许联系。集成学习通过训练多个模型来提升性能。Dropout 有点类似于（但又不完全等同于）训练了多个模型。

  同时，我们也可把 dropout 理解为一个增强稳健性（robustness）的方法：删除一部分神经元意味着丢掉一部分信息，通过 dropout 训练使得网络对于这种部分信息的丢失不那么敏感。

用 Dropout 防止过拟合

- Dropout 是 2012 年被首次提出的。

  ImageNet Classification with Deep Convolutional Neural Networks, by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton (2012).

  这篇文章通过 dropout 加上一个改进过的 L2 正则化，把 MNIST 手写数字识别的准确率从 98.4% 提升到了 98.7%

在 PyTorch 中使用 Dropout

- **PyTorch 中可以直接使用 Dropout 层构建网络**

DROPOUT

CLASS torch.nn.Dropout(*p=0.5, inplace=False*) [SOURCE]

During training, randomly zeroes some of the elements of the input tensor with probability `p` using samples from a Bernoulli distribution. Each channel will be zeroed out independently on every forward call.

This has proven to be an effective technique for regularization and preventing the co-adaptation of neurons as described in the paper Improving neural networks by preventing co-adaptation of feature detectors .

Furthermore, the outputs are scaled by a factor of $\frac{1}{1-p}$ during training. This means that during evaluation the module simply computes an identity function.

Parameters:

- **p** (*float*) – probability of an element to be zeroed. Default: 0.5
- **inplace** (*bool*) – If set to `True`, will do this operation in-place. Default: `False`

Shape:

- Input: $(*)$. Input can be of any shape
- Output: $(*)$. Output is of the same shape as input

在 PyTorch 中使用 Dropout

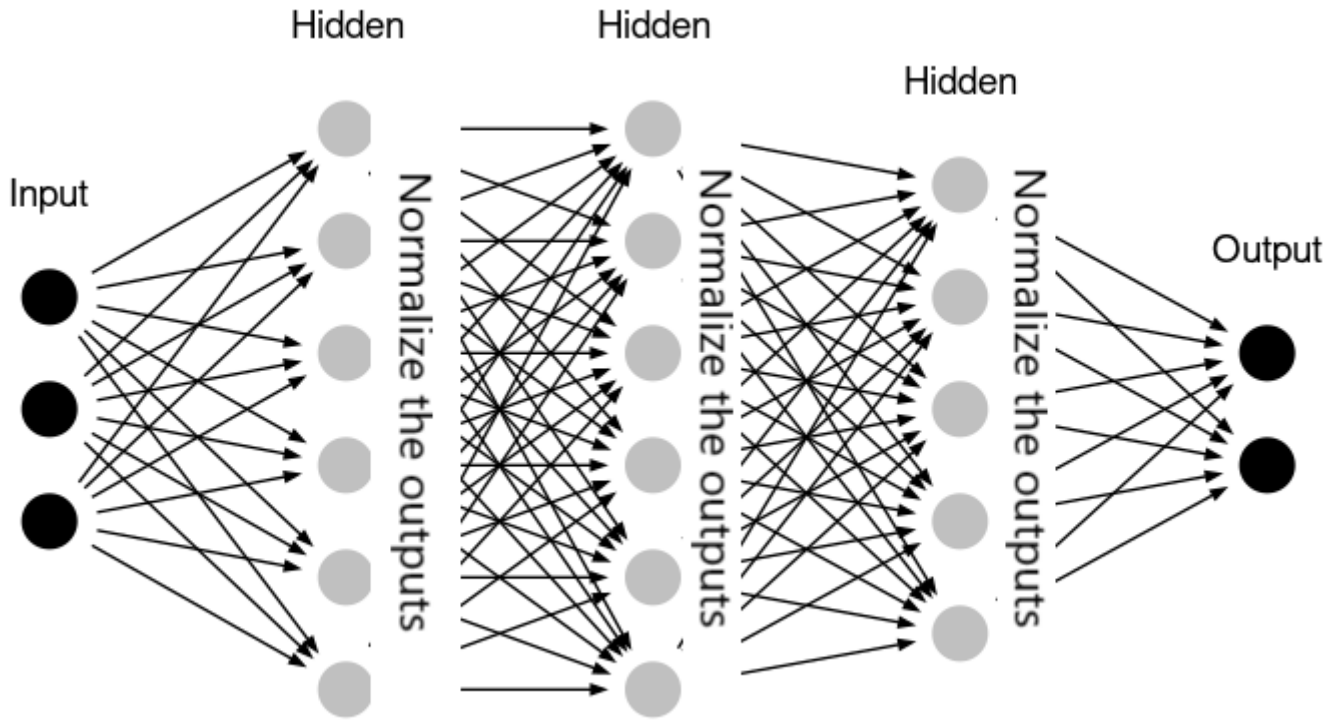
- PyTorch 中可以直接使用 Dropout 层构建网络

```
class NeuralNet(nn.Module):
    def __init__(self):
        ……
        self.Dropout = nn.Dropout(0.5)
        ……

    def forward(self, x):
        ……
        out = self.Dropout(out)
        ……
        return out
```

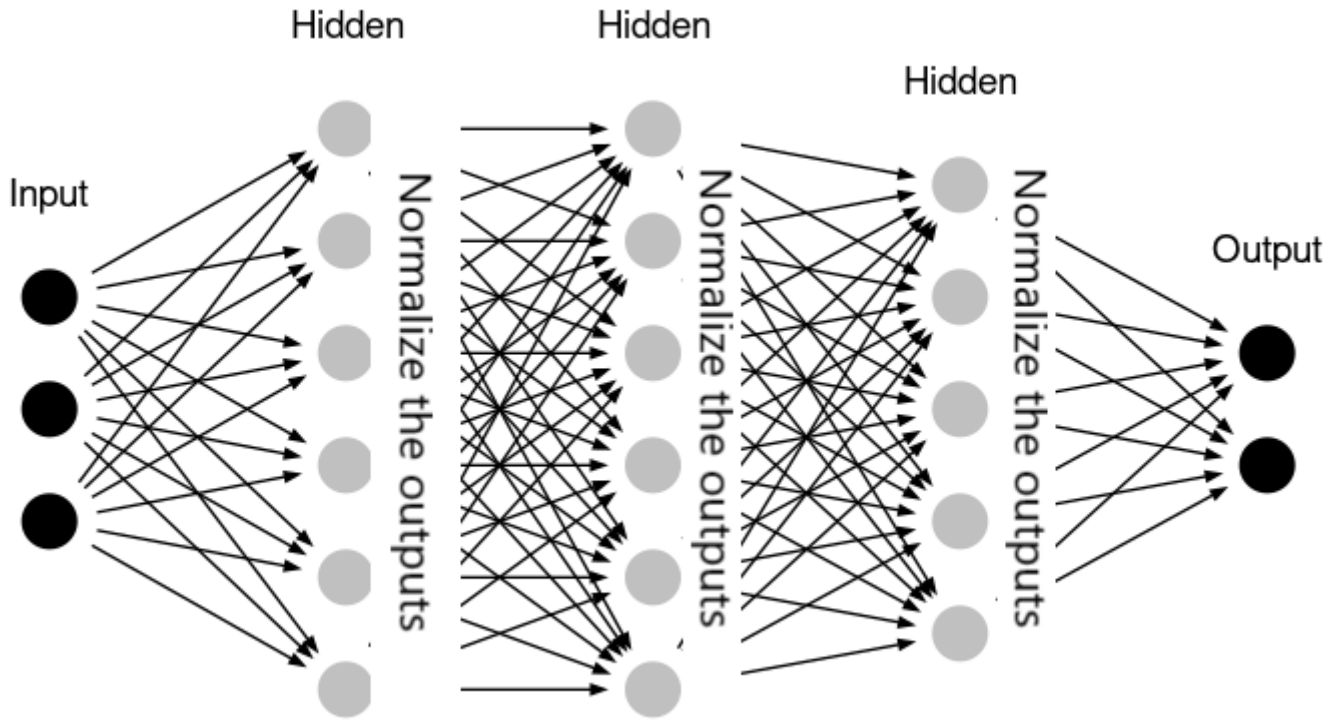
用批标准化（Batch Normalization）防止过拟合

- **近年来，Batch normalization 基本取代了 dropout，成为更常用的防止过拟合的方法**



用批标准化（Batch Normalization）防止过拟合

- **Batch normalization：将一整层的输入（向量）减去各元素的均值，在除以标准差**



用批标准化（Batch Normalization）防止过拟合

- **Batch normalization 可以防止过拟合**
  - 它让所有信号都有机会下一层，所有参数都可能“进一步学习”，抑制了极端情况

- **Batch normalization 通常不能与 dropout 同时使用**

在 PyTorch 中使用批标准化

- [torch.nn — PyTorch 1.13 documentation]

TORCH.NN

These are the basic building blocks for graphs:

torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization

Normalization Layers

`nn.BatchNorm1d`	Applies Batch Normalization over a 2D or 3D input as described in the paper Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift .
`nn.BatchNorm2d`	Applies Batch Normalization over a 4D input (a mini-batch of 2D inputs with additional channel dimension) as described in the paper Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift .
`nn.BatchNorm3d`	Applies Batch Normalization over a 5D input (a mini-batch of 3D inputs with additional channel dimension) as described in the paper Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift .
`nn.LazyBatchNorm1d`	A `torch.nn.BatchNorm1d` module with lazy initialization of the `num_features` argument of the `BatchNorm1d` that is inferred from the `input.size(1)`.
`nn.LazyBatchNorm2d`	A `torch.nn.BatchNorm2d` module with lazy initialization of the `num_features` argument of the `BatchNorm2d` that is inferred from the `input.size(1)`.
`nn.LazyBatchNorm3d`	A `torch.nn.BatchNorm3d` module with lazy initialization of the `num_features` argument of the `BatchNorm3d` that is inferred from the `input.size(1)`.

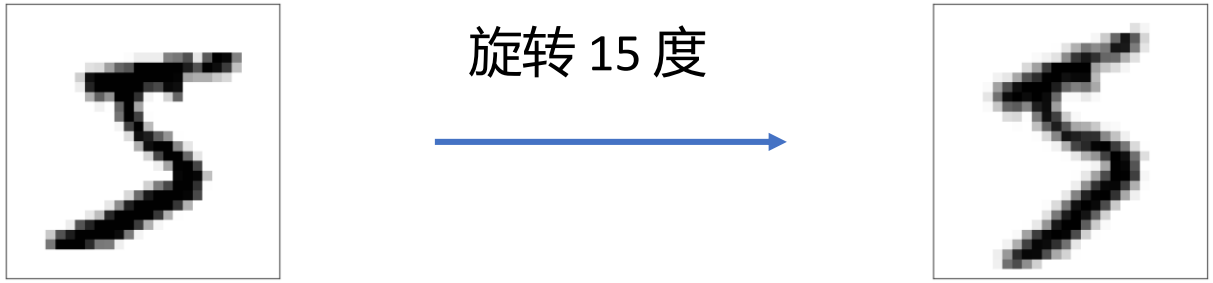
数据增强

- 训练数据样本少或者多样性差，也会导致过拟合。解决的办法之一，是利用已有的样本，人为地创造一些新的样本

- 比如，我们可以对图像数据样本进行旋转、拉伸、镜像等等，得到新的样本

数据增强

- 例： MNIST 训练样本中的数字 5



- 类似地，对语音识别数据，我们可以稍稍加快或减慢速度，或者加入少量噪音，得到新的样本

权重初始化

使用高斯分布进行权重初始化

- 之前我们提到，在开始训练神经网络之前，我们选择随机设置权重 w 和偏置 b 的初始值，比如用均值为 0、标准差为 1 的独立高斯分布随机变量。

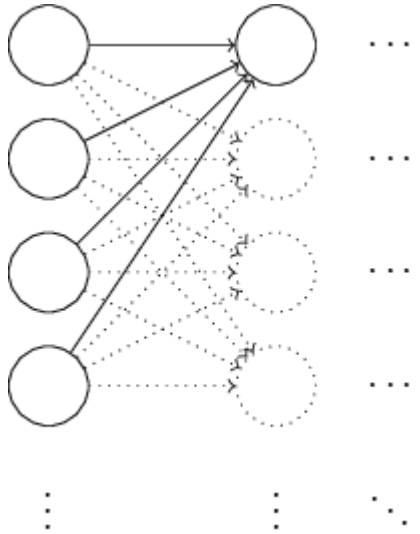
  现在我们来分析一下，有没有可能使用其它的初始化方法，使神经网络训练的效果更好。

  事实上，我们可以使用一个新的初始化方法，大幅改善神经网络训练的收敛。

权重初始化：可能出现的问题

- 考虑一层输入有 1000 个神经元的网络。假设我们使用均值为 0、标准差为 1 的高斯分布对参数进行初始化。

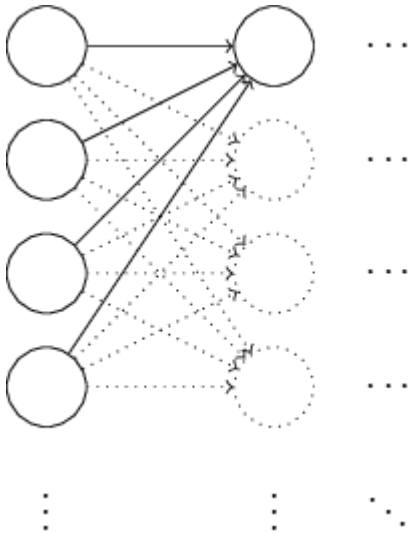
  我们来分析一下此时第一个隐藏层中第一个神经元的情况。



权重初始化：可能出现的问题

- 我们假设前一层输入到本隐藏层的信号向量 x 有一半的元素为 0，一半的元素为 1。虽然这只是一个特殊情况，但从这个例子中我们可以得到一些可以推广的思路。

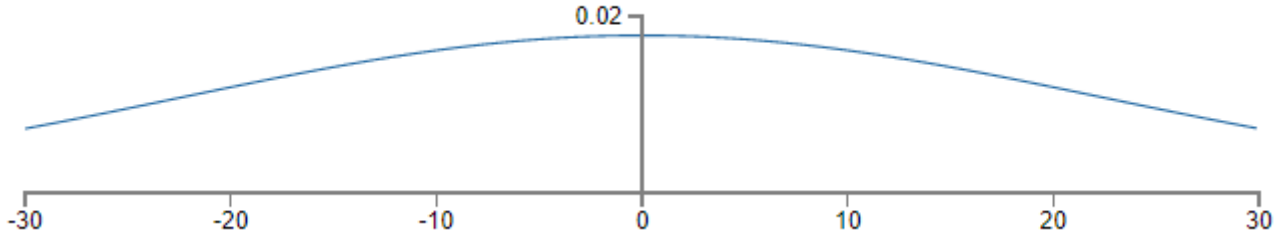
  这个神经元的加权输入是 $z = \sum_j w_j x_j + b$



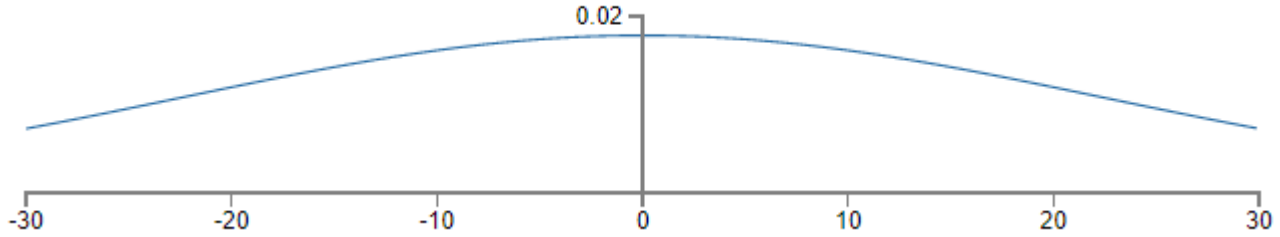
权重初始化：可能出现的问题

- $z = \sum_j w_j x_j + b$ 中，有500个 x_j 是0，500个 x_j 是 1。所以 z 就是 501 个独立的正态分布的随机变量的和（500个 w_j 和 1 个 b）。

  根据正态分布的性质，z 是一个均值为 0、标准差为 $\sqrt{501} \approx 22.4$ 的正态分布的随机变量。也就是说，z 的分布很“平坦”，如下图。



权重初始化：可能出现的问题



- 可以看出，$z \in [-1, 1]$ 的概率是很小的。那么接下来，在使用 Sigmoid 激活函数的情况下，$\sigma(z)$ 很可能会饱和，即输出接近 0 或者 1。

  在明显饱和的情况下，这个神经元的参数在训练中学习的速率就会很慢。

权重初始化：解决方案

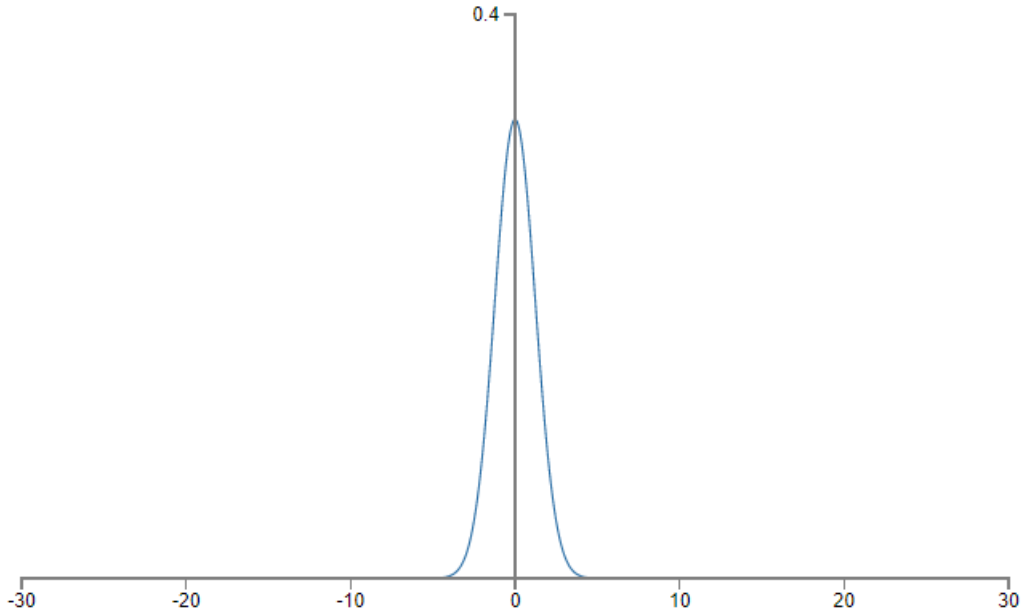
- 如何使 $z = \sum_j w_j x_j + b$ 在初始化后不那么容易饱和呢？

  我们可以将初始权重 w_j 设为均值为 0 、标准差为 $\frac{1}{\sqrt{n_{in}}}$ 的高斯随机变量，其中 n_{in} 是输入到这个神经元的连接数。

  对偏置 b，我们依然使用均值为 0、标准差为 1 的高斯随机数。

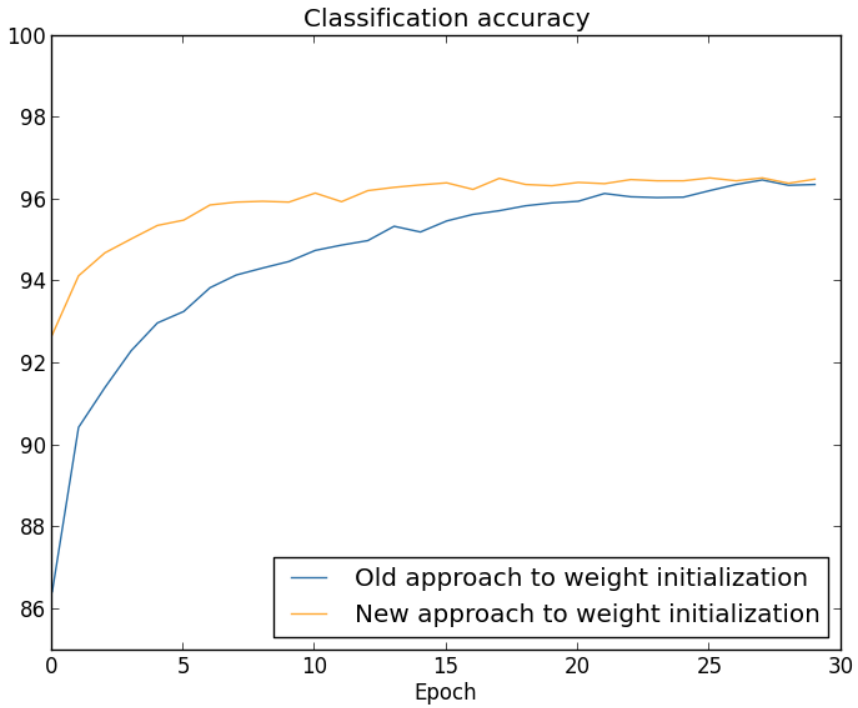
权重初始化：解决方案

- 将初始权重的标准差减小为 $\frac{1}{\sqrt{n_{in}}}$ 之后，神经元中 $z = \sum_j w_j x_j + b$ 的概率分布将更加“集中”，出现饱和的概率大大降低



权重初始化：效果

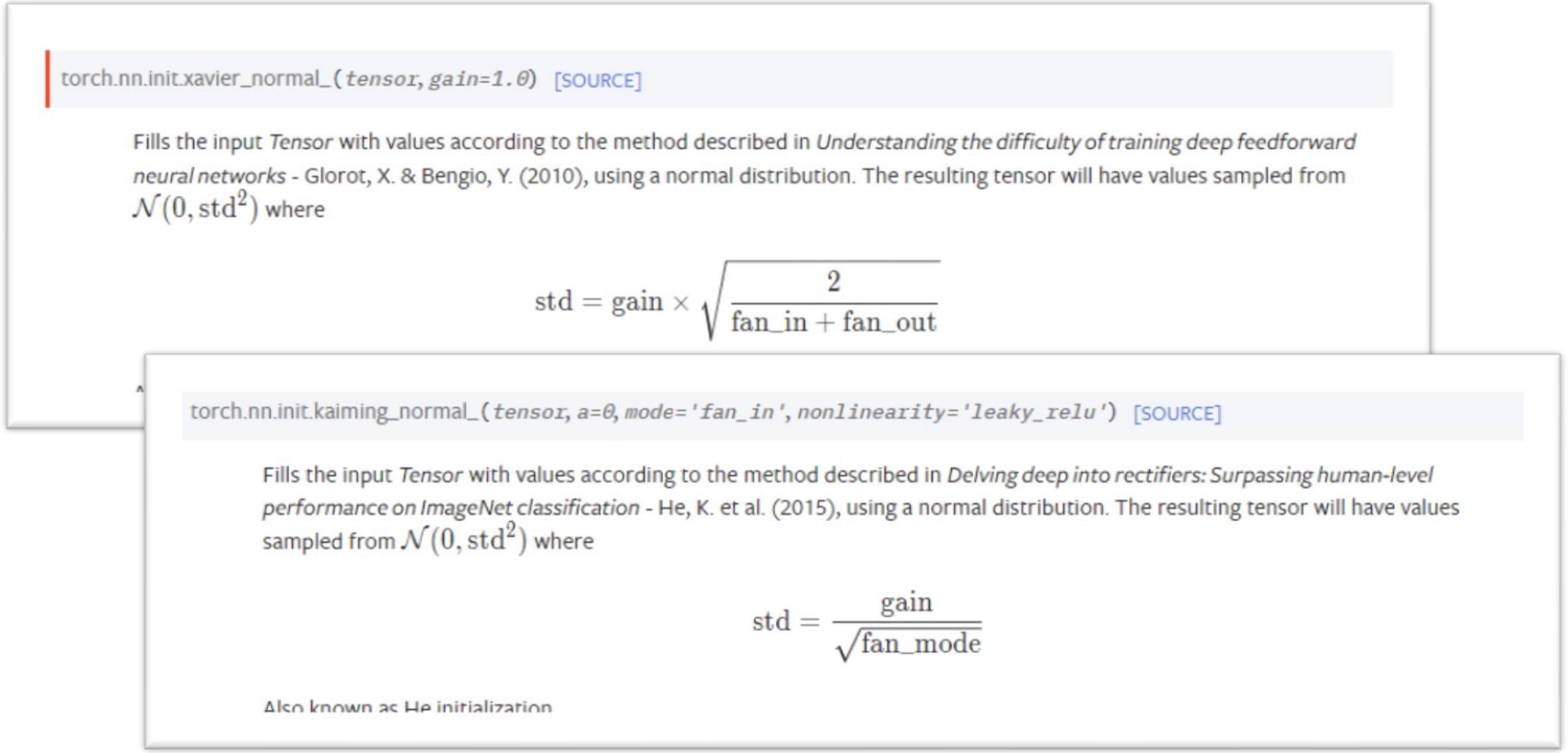
- 对于 MNIST 手写数字识别问题，我们使用一个含 30 个神经元的隐藏层，mini-batch 大小为 10，交叉熵代价函数，以及 $\lambda = 5$ 的正则化项



使用新的初始化方法，模型训练收敛更快

在 PyTorch 中使用权重初始化

- **PyTorch 中常用的权重初始化有以下两类**
  - Xavier
  - Kaiming（在 ReLU 激活函数下可能表现比 Xavier 好）



torch.nn.init.xavier_normal_(*tensor*, *gain=1.0*) [SOURCE]

Fills the input *Tensor* with values according to the method described in *Understanding the difficulty of training deep feedforward neural networks* - Glorot, X. & Bengio, Y. (2010), using a normal distribution. The resulting tensor will have values sampled from $\mathcal{N}(0, \text{std}^2)$ where

$$\text{std} = \text{gain} \times \sqrt{\frac{2}{\text{fan_in} + \text{fan_out}}}$$

torch.nn.init.kaiming_normal_(*tensor*, *a=0*, *mode='fan_in'*, *nonlinearity='leaky_relu'*) [SOURCE]

Fills the input *Tensor* with values according to the method described in *Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification* - He, K. et al. (2015), using a normal distribution. The resulting tensor will have values sampled from $\mathcal{N}(0, \text{std}^2)$ where

$$\text{std} = \frac{\text{gain}}{\sqrt{\text{fan_mode}}}$$

Also known as He initialization

在 PyTorch 中使用权重初始化

- **PyTorch 中使用权重初始化**

```
class NeuralNet(nn.Module):
    def __init__(self):
        super(NeuralNet, self).__init__()
        self.fc1 = nn.Linear(784, 15)
        nn.init.xavier_normal_(self.fc1.weight)
        self.sigmoid = nn.Sigmoid()
        self.fc2 = nn.Linear(15, 10)
        nn.init.xavier_normal_(self.fc2.weight)

    def forward(self, x):
        out = self.fc1(x)
        out = self.sigmoid(out)
        out = self.fc2(out)
        return out
```

练习题

在神经网络训练中，权重初始化的主要作用是_____

A. 减少神经网络中参数的数量

B. 抑制神经网络过拟合

C. 加快神经网络训练中参数的收敛速度

D. 提升神经网络在分类任务中的效果

练习题

在神经网络训练中，权重初始化的主要作用是_____

A. 减少神经网络中参数的数量

B. 抑制神经网络过拟合

C. 加快神经网络训练中参数的收敛速度

D. 提升神经网络在分类任务中的效果

【答案】：C

【解析】：权重初始化的核心目的是为梯度下降提供一个良好的起点，
加快收敛速度，并避免训练过程中出现梯度消失或梯度爆炸问题。

超参数
(Hyper-parameters)

参数与超参数

- 一般来讲，我们常说的“参数”是指模型中那些需要训练的变量，如权重 w 和偏置 b

  而“超参数”是指描述模型结构的变量（如神经网络模型的隐藏层数，每层中神经元的数量），以及描述模型学习方法的变量（如学习率 η，mini-batch的大小、正则化参数 λ 等）

  可以认为，除了“参数”权重 w 和偏置 b，在设计、训练神经网络模型的过程中我们需要确定的变量，都属于“超参数”

超参数

- 以下都可以看作神经网络模型的超参数
  - 学习速率 η
  - 正则化参数 λ
  - 神经网络层数 L
  - 每一个隐藏层中神经元的个数
  - 学习的回合数 Epoch
  - 代价函数的选择

超参数（续上页）

- 以下都可以看作神经网络模型的超参数
  - Mini-batch 的大小
  - 输出神经元的编码方式
  - 权重初始化的方法
  - 激活函数的种类
  - 参加训练模型的数据规模
  - ……

超参数的影响

- 对于 MNIST 手写数字识别问题，我们使用一个含 30 个神经元的隐藏层，mini-batch 大小为 10，交叉熵代价函数

  假如我们使用学习速率 $\eta = 10.0$，正则化参数 $\lambda = 1000$，则训练二十多个 epoch 之后，模型的准确率只有 10% 左右

  和随机猜测的结果一样，模型完全没有效果!

超参数

- **对于一个给定的神经网络机器学习问题，寻找最优的超参数，是个十分困难的**
- **实际使用中，我们能用的经验策略，总结起来主要有以下几点**
  - 对于一个新问题，找已有人解决过的类似问题的超参数
  - 用简化的、快捷的问题，来快速迭代尝试
    - 了解超参数对模型效果的大致影响

确定超参数范围：使用简化问题

- **如何设计简化、快捷的问题?**

- **以 MNIST 手写数字识别为例。假如我们完全不知道超参数的大致范围，我们可以设计以下问题**
  - 仅使用 MNIST 数据集中的 0 和 1 图片
  - 假如我们觉得有一层隐藏层的网络会好用（如神经元数为[784,30,10]的网络，我们可以先试着训练一个无隐藏层的网络（[784,10]），用这个无隐藏层的网络来寻找合适超参数的大致范围
- **对于简化的问题，只要最终的神经网络效果明显优于随机猜测，我们就找到了超参数的大致范围**

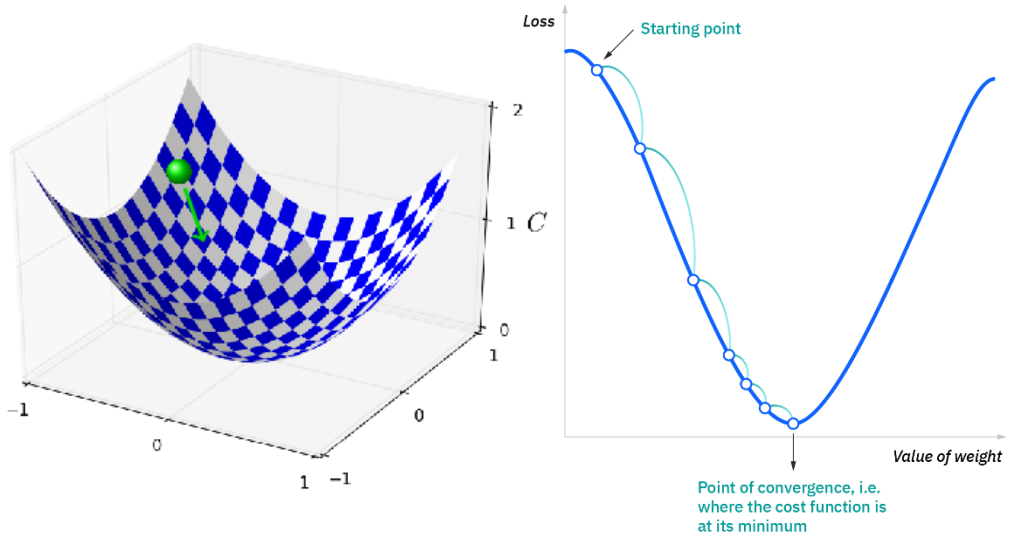
超参数：学习速率

- 学习速率 η 直接关系到训练的效果是否收敛

回顾：学习速率 η

$$\boldsymbol{w} \rightarrow \quad \boldsymbol{w}' = \boldsymbol{w} - \eta \frac{\partial C}{\partial \boldsymbol{w}}$$

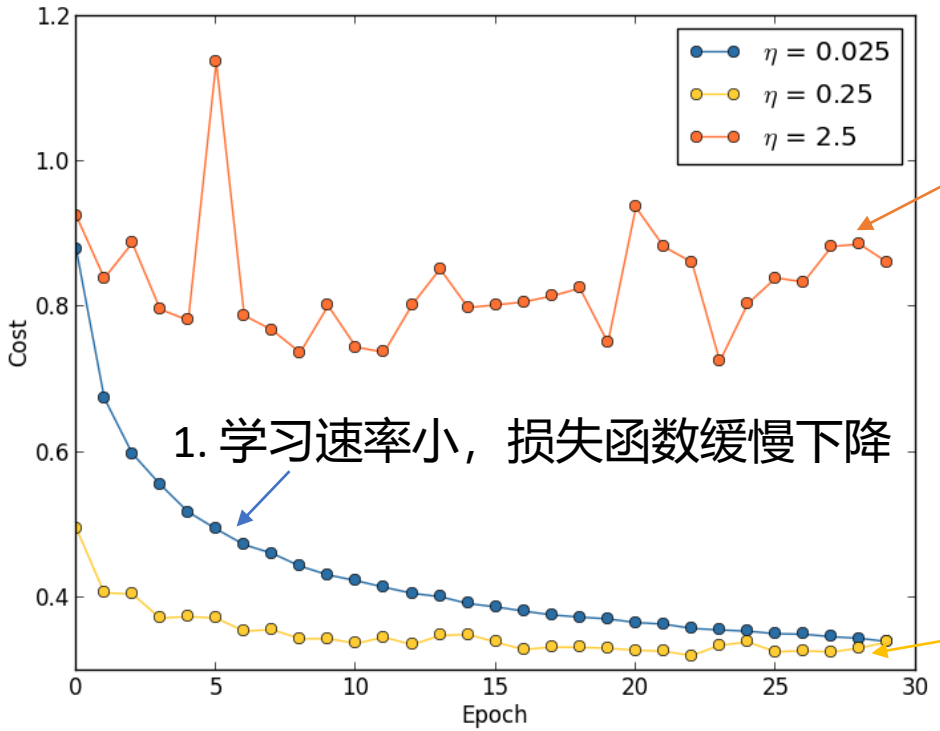
$$b \rightarrow \quad b' = b - \eta \frac{\partial C}{\partial b}$$



学习速率太小或太大均不利于参数收敛

超参数：学习速率

- 对于 MNIST 手写数字识别问题，我们采用 $\eta = 0.025$，$\eta = 0.25$，$\eta = 2.5$ 三种学习速率进行训练，其收敛的效果如下图



1. 学习速率小，损失函数缓慢下降

2. 学习速率适中，损失函数很快下降并接近饱和（接近最优），开始震荡

3. 学习速率大，损失函数几乎一直在震荡，效果不佳

超参数：学习回合数 epoch

- 一般来说，如果训练效果在最近的 n 个回合（epoch）中都没有显著提升，可以考虑终止训练

  一般可以取 $n = 10$，也可考虑 $n = 5, 20, ...$

超参数：正则化参数

- 回顾：正则化参数 λ 可以防止过拟合

$$C = C_0 + \frac{\lambda}{2n} \sum_w w^2$$

可以考虑首先使用不含正则化项的代价函数，调节其它超参数

超参数：正则化参数

- 粗调节 λ 时，每次增大或减小 10 倍；当验证集上的效果与训练集接近后（即没有显著的过拟合现象之后）再精调 λ

一般来说，调节 λ 之后，可以考虑重新调节学习速率 η

自动调节超参数的技术

- 使用基于数据等方法，设定程序来自动调节神经网络模型训练中的超参数，也是当前深度学习研究的方向之一